earth-science-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `earth_science_ai999x.about()` for the same answer.

Every capability earth-science-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import earth_science_ai999x

result = earth_science_ai999x.solve('potential_temperature', T='298.15 K', p='85000 Pa')
result.show()
# theta = 312.321 K
```

By topic

topic	capabilities
Applied meteorology	Wind chill temperature index, Heat index (NWS)
Atmospheric and ocean dynamics	Period of inertial oscillations
Atmospheric dynamics	Coriolis parameter, Wind components from speed and direction, Wind speed and direction from components, Geostrophic wind from a height gradient, Brunt-Väisälä (buoyancy) frequency, Phase speed of a barotropic Rossby wave, Gradient wind in curved flow, Rossby number
Atmospheric moisture	Saturation vapour pressure over water, Relative humidity from temperature and dewpoint, Dewpoint from vapour pressure, Dewpoint from temperature and relative humidity, Water-vapour mixing ratio from vapour pressure, Saturation mixing ratio, Specific humidity from dewpoint and pressure
Atmospheric thermodynamics	Potential temperature, Temperature from potential temperature, Temperature of a dry parcel after adiabatic ascent, Virtual temperature, Density of moist air, Moist static energy, Lifting condensation level (Romps 2017), Equivalent potential temperature (Bolton 1980), Saturated (moist) adiabatic lapse rate, Pressure at height in an isothermal atmosphere, Atmospheric scale height, Thickness of a layer (hypsometric equation), Pressure at height with a constant lapse rate, Pressure at a height in the standard atmosphere, Height of a pressure level in the standard atmosphere, Geopotential and geopotential height
Climate	Warming of a one-box energy-balance model
Climate and radiation	Radiative forcing of a change in CO2 (Myhre et al. 1998), Solar elevation and azimuth
Geochronology	Fraction of a radioactive parent remaining, Radiometric age from a daughter/parent ratio, Potassium-argon age (branched decay), Isochron age (e.g. Rb-Sr) from sample ratios, Conventional radiocarbon age
Geodynamics	Mountain root by Airy isostasy, Column density by Pratt isostasy

topic	capabilities
Geomagnetism	Inclination and strength of a geocentric dipole field, Paleolatitude from magnetic inclination
Geomorphology	Stream power of a river
Geophysics	Gravitational acceleration at a latitude and height, Gravity anomaly of an infinite horizontal slab (Bouguer), Gravity anomaly above a buried sphere
Geothermics	Steady geotherm with radiogenic heat production, Surface heat flow of cooling oceanic lithosphere (half-space model)
Hydrogeology	Groundwater discharge by Darcy's law
Ocean dynamics	Speed of a shallow-water (long) wave and tsunami travel time, Ekman layer depth, Ekman transport
Oceanography	Density of pure water (SMOW, Bigg 1967), Density of seawater at the surface (UNESCO 1981), In-situ density of seawater (EOS-80), Freezing point of seawater (UNESCO 1983), Speed of sound in seawater (Chen and Millero 1977), Sea pressure at a depth (Saunders 1981), Depth from sea pressure (Fofonoff and Millard 1983), Adiabatic temperature gradient of seawater (Bryden 1973), Potential temperature of seawater (Fofonoff 1977), Water-wave dispersion at finite depth
Plate tectonics	Plate velocity from an Euler pole, Depth of the ocean floor with age (half-space cooling)
Radiation balance	Radiant flux emitted by a black or grey body (Stefan-Boltzmann), Wavelength of peak blackbody emission (Wien), Planetary equilibrium (effective radiating) temperature, Surface temperature in the single-layer greenhouse model, Surface temperature with N fully absorbing layers, Daily mean insolation at the top of the atmosphere
Sedimentology	Settling velocity of a small grain (Stokes)
Seismology	P- and S-wave speeds from bulk and shear moduli, P- and S-wave speeds from Young's modulus and Poisson's ratio, Elastic moduli and Poisson's ratio from Vp, Vs and density, Epicentral distance on a spherical Earth, Geodesic distance and azimuths on the WGS84 ellipsoid (Vincenty), Moment magnitude from seismic moment, Local (Richter) magnitude from Wood-Anderson amplitude, Gutenberg-Richter b-value (maximum likelihood), Normal-moveout velocity from reflection picks, Distance to an earthquake from the S-P interval

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
potential_temperature	Potential temperature	T, p	theta
temperature_from_potential_temperature	Temperature from potential temperature	theta, p	T
dry_adiabatic_ascent	Temperature of a dry parcel after adiabatic ascent	T, dz	T2, lapse_rate
saturation_vapor_pressure	Saturation vapour pressure over water	T	es
relative_humidity_from_dewpoint	Relative humidity from temperature and dewpoint	T, Td	RH, RH_percent
dewpoint_from_vapor_pressure	Dewpoint from vapour pressure	e	Td
dewpoint_from_relative_humidity	Dewpoint from temperature and relative humidity	T, RH	Td
mixing_ratio	Water-vapour mixing ratio from vapour pressure	e, p	w, w_g_per_kg
saturation_mixing_ratio	Saturation mixing ratio	p, T	ws, ws_g_per_kg
virtual_temperature	Virtual temperature	T, w	Tv
moist_air_density	Density of moist air	p, T, w	rho, Tv

id	operation	inputs	outputs
specific_humidity_from_dewpoint	Specific humidity from dewpoint and pressure	p, Td	q, w
moist_static_energy	Moist static energy	z, T, q	h, s
lifting_condensation_level	Lifting condensation level (Romps 2017)	p, T, Td	p_lcl, T_lcl
equivalent_potential_temperature	Equivalent potential temperature (Bolton 1980)	p, T, Td	theta_e
moist_adiabatic_lapse_rate	Saturated (moist) adiabatic lapse rate	T, p	lapse_rate, rs
wind_chill	Wind chill temperature index	T, V	WCT
heat_index	Heat index (NWS)	T, RH	HI
coriolis_parameter	Coriolis parameter	lat	f
inertial_period	Period of inertial oscillations	lat	T_inertial, T_hours, f
wind_components	Wind components from speed and direction	speed, direction	u, v
wind_speed_direction	Wind speed and direction from components	u, v	speed, direction
geostrophic_wind	Geostrophic wind from a height gradient	lat, dz, dn	Vg, f
brunt_vaisala_frequency	Brunt-Väisälä (buoyancy) frequency	theta, dtheta_dz	N, period
isothermal_pressure	Pressure at height in an isothermal atmosphere	p0, T, z	p, H
scale_height	Atmospheric scale height	T	H
hypsometric_thickness	Thickness of a layer (hypsometric equation)	p1, p2, Tv	dz
pressure_with_lapse_rate	Pressure at height with a constant lapse rate	p0, T0, gamma, z	p, T
standard_atmosphere_pressure	Pressure at a height in the standard atmosphere	z	p
standard_atmosphere_height	Height of a pressure level in the standard atmosphere	p	z
geopotential_height	Geopotential and geopotential height	z	phi, Z
rossby_wave_speed	Phase speed of a barotropic Rossby wave	U, lat, Lx	c, beta, Ls
blackbody_flux	Radiant flux emitted by a black or grey body (Stefan-Boltzmann)	T, eps	F
wien_peak_wavelength	Wavelength of peak blackbody emission (Wien)	T	lambda_max, lambda_um
planetary_equilibrium_temperature	Planetary equilibrium (effective radiating) temperature	S, albedo	Te, absorbed
greenhouse_single_layer	Surface temperature in the single-layer greenhouse model	S, albedo, eps	Ts, Ta

id	operation	inputs	outputs
greenhouse_n_layers	Surface temperature with N fully absorbing layers	S, albedo, N	Ts, T_top, T_bottom_layer
daily_mean_insolation	Daily mean insolation at the top of the atmosphere	lat, decl, S0	Q, day_length_h
energy_balance_response	Warming of a one-box energy-balance model	F, lam, C, t	dT, dT_eq, tau_years
pure_water_density	Density of pure water (SMOW, Bigg 1967)	T	rho
seawater_density_surface	Density of seawater at the surface (UNESCO 1981)	S, T	rho, sigma_t
seawater_density	In-situ density of seawater (EOS-80)	S, T, p	rho, sigma
seawater_freezing_point	Freezing point of seawater (UNESCO 1983)	S, p	Tf, Tf_C
sound_speed_seawater	Speed of sound in seawater (Chen and Millero 1977)	S, T, p	c
ocean_pressure_from_depth	Sea pressure at a depth (Saunders 1981)	depth, lat	p, p_dbar
ocean_depth_from_pressure	Depth from sea pressure (Fofonoff and Millard 1983)	p, lat	depth
seawater_adiabatic_gradient	Adiabatic temperature gradient of seawater (Bryden 1973)	S, T, p	gamma, gamma_per_km_dbar
seawater_potential_temperature	Potential temperature of seawater (Fofonoff 1977)	S, T, p, pr	theta, theta_C, dtheta
gravity_at_latitude	Gravitational acceleration at a latitude and height	lat, z	g
shallow_water_wave	Speed of a shallow-water (long) wave and tsunami travel time	h, distance	c, travel_time, travel_time_h
ekman_depth	Ekman layer depth	Az, lat	D_E, delta
ekman_transport	Ekman transport	tau, lat, rho	M, mass_transport
seismic_velocities_from_moduli	P- and S-wave speeds from bulk and shear moduli	K, mu, rho	Vp, Vs, Vp_Vs
seismic_velocities_from_young	P- and S-wave speeds from Young's modulus and Poisson's ratio	E, nu, rho	Vp, Vs, Vp_Vs
elastic_constants_from_velocities	Elastic moduli and Poisson's ratio from Vp, Vs and density	Vp, Vs, rho	mu, lam, K, E, nu
epicentral_distance	Epicentral distance on a spherical Earth	lat1, lon1, lat2, lon2	delta, distance_km
geodesic_distance_azimuth	Geodesic distance and azimuths on the WGS84 ellipsoid (Vincenty)	lat1, lon1, lat2, lon2	distance, azimuth, back_azimuth
moment_magnitude	Moment magnitude from seismic moment	M0	Mw, M0_dyne_cm
local_magnitude	Local (Richter) magnitude from Wood-Anderson amplitude	A, r	ML

id	operation	inputs	outputs
gutenberg_richter_b_value	Gutenberg-Richter b-value (maximum likelihood)	mags, Mc, dM	b, a, n, mean
nmo_velocity	Normal-moveout velocity from reflection picks	x, t	V_nmo, t0, depth
dipole_inclination	Inclination and strength of a geocentric dipole field	lat	inclination, F_ratio
paleolatitude	Paleolatitude from magnetic inclination	I_deg	paleolatitude, pole_distance
plate_velocity	Plate velocity from an Euler pole	pole_lat, pole_lon, omega, lat, lon	speed, azimuth
bouguer_slab	Gravity anomaly of an infinite horizontal slab (Bouguer)	drho, h	dg_mGal, dg
buried_sphere_anomaly	Gravity anomaly above a buried sphere	drho, R, z	dg_mGal, dg
geotherm_radiogenic	Steady geotherm with radiogenic heat production	T0, q0, k, A, z	T, q
seafloor_depth_age	Depth of the ocean floor with age (half-space cooling)	t, dr, Tm, Tw, alpha, kappa, rho_m, rho_w	depth, subsidence
radioactive_decay_remaining	Fraction of a radioactive parent remaining	half_life, t	fraction, decayed, lambda
radiometric_age	Radiometric age from a daughter/parent ratio	ratio, half_life	age, lambda
potassium_argon_age	Potassium-argon age (branched decay)	ratio, lam, lam_e	age
isochron_age	Isochron age (e.g. Rb-Sr) from sample ratios	x, y, lam	age, slope, initial_ratio
radiocarbon_age	Conventional radiocarbon age	F	age
stokes_settling_velocity	Settling velocity of a small grain (Stokes)	d, rho_s, rho_f, mu	w, Re
co2_radiative_forcing	Radiative forcing of a change in CO2 (Myhre et al. 1998)	C, C0, lam	dF, doublings, dT_eq
sp_epicentral_distance	Distance to an earthquake from the S-P interval	dt, vp, vs	distance, distance_km, tp, ts
airy_isostasy	Mountain root by Airy isostasy	h, rho_c, rho_m, tc	root, root_km, crust_thickness
pratt_isostasy	Column density by Pratt isostasy	h, rho0, D	rho, drho
gradient_wind	Gradient wind in curved flow	lat, R, Vg, flow	V, ratio, f
wave_dispersion	Water-wave dispersion at finite depth	T, h	L, c, cg, kh, L0
darcy_flow	Groundwater discharge by Darcy's law	K, A, dh, L, n	Q, q, gradient, v
stream_power	Stream power of a river	Q, S, w, rho	Omega, omega
rossby_number	Rossby number	U, L, lat	Ro, f
solar_elevation_angle	Solar elevation and azimuth	lat, decl, H	elevation, zenith, azimuth

id	operation	inputs	outputs
half_space_heat_flow	Surface heat flow of cooling oceanic lithosphere (half-space model)	t, dT, k, kappa	q, q_mW, thickness

Potential temperature — `potential_temperature`

The temperature an air parcel would have if brought dry-adiabatically to 1000 hPa (Poisson's equation).

$$\theta = T\left(\frac{p_0}{p}\right)^{R_d/c_{pd}}, \quad p_0 = 1000\ \mathrm{hPa}$$

input	meaning	condition
`T`	air temperature	
`p`	air pressure	

output	meaning
`theta`	potential temperature (K)

- **Method:** Poisson's equation with $\kappa = R_d/c_pd$
- **Assumes:** 150 $K \leq T \leq 350$ K; 1 hPa <= p <= 1100 hPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('potential_temperature', T=298.15, p=85000.0)
# -> theta = 312.321 K   (SOLVED)
```

```
earth_science_ai999x.solve('potential_temperature', T=263.15, p=50000.0)
# -> theta = 320.783 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Potential temperature with temperature = 25 °C and pressure = 850 hPa.	SOLVED	theta = 312.321 K
Potential temperature with temperature = -10 °C and pressure = 500 hPa.	SOLVED	theta = 320.783 K
Potential temperature with temperature = 25 °C.	NEEDS_INPUT	potential_temperature needs p (air pressure). Example: solve('potential_temperature', T='298.15 K', p='85000 Pa').
Potential temperature with temperature = 25 °C and pressure = 850 hPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Temperature from potential temperature — `temperature_from_potential_temperature`

The air temperature at pressure p of a parcel with a given potential temperature.

$$T = \theta \left(\frac{p}{p_0} \right)^{R_d / c_{pd}}$$

input	meaning	condition
`theta`	potential temperature	
`p`	air pressure	

output	meaning
`T`	air temperature (K)

- **Method:** Poisson's equation inverted
- **Assumes:** 150 $K \leq \theta \leq 600$ K; 1 hPa <= p <= 1100 hPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('temperature_from_potential_temperature', theta=300.0, p=70000.0)
# -> T = 270.934 K   (SOLVED)
```

```
earth_science_ai999x.solve('temperature_from_potential_temperature', theta=330.0, p=25000.0)
# -> T = 222.074 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Temperature from potential temperature with potential temperature = 300 K and pressure = 700 hPa.	SOLVED	T = 270.934 K
Temperature from potential temperature with potential temperature = 330 K and pressure = 250 hPa.	SOLVED	T = 222.074 K
Temperature from potential temperature with potential temperature = 300 K.	NEEDS_INPUT	temperature_from_potential_temperature needs p (air pressure). Example: solve('temperature_from_potential_temperature',
Temperature from potential temperature with potential temperature = 300 K and pressure = 700 hPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Temperature of a dry parcel after adiabatic ascent — `dry_adiabatic_ascent`

The temperature of an unsaturated air parcel lifted dry-adiabatically through a height dz, and the dry adiabatic lapse rate g/c_pd.

$$T_2 = T - \Gamma_d \, \Delta z, \quad \Gamma_d = g / c_{pd}$$

input	meaning	condition
`T`	starting temperature of the parcel	
`dz`	height through which the parcel rises	

output	meaning
`T2`	parcel temperature after the ascent (K)
`lapse_rate`	dry adiabatic lapse rate g/c_pd (K/m)

- **Method:** dry adiabatic lapse rate g/c_pd times the height gained
- **Assumes:** $0 < dz \leq 20$ km; the parcel stays unsaturated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('dry_adiabatic_ascent', T=293.15, dz=1500.0)
# -> T2 = 278.508 K, lapse_rate = 0.0097611 K/m   (SOLVED)
```

```
earth_science_ai999x.solve('dry_adiabatic_ascent', T=303.15, dz=800.0)
# -> T2 = 295.341 K, lapse_rate = 0.0097611 K/m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dry adiabatic ascent with temperature = 20 °C and height gain = 1500 m.	SOLVED	T2 = 278.508 K, lapse_rate = 0.0097611 K/m
Dry adiabatic ascent with temperature = 30 °C and height gain = 800 m.	SOLVED	T2 = 295.341 K, lapse_rate = 0.0097611 K/m
Dry adiabatic ascent with temperature = 20 °C.	NEEDS_INPUT	dry_adiabatic_ascent needs dz (height through which the parcel rises). Example: solve('dry_adiabatic_ascent', T='293.15
Dry adiabatic ascent with temperature = 20 °C and height gain = 1500 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Saturation vapour pressure over water — `saturation_vapor_pressure`

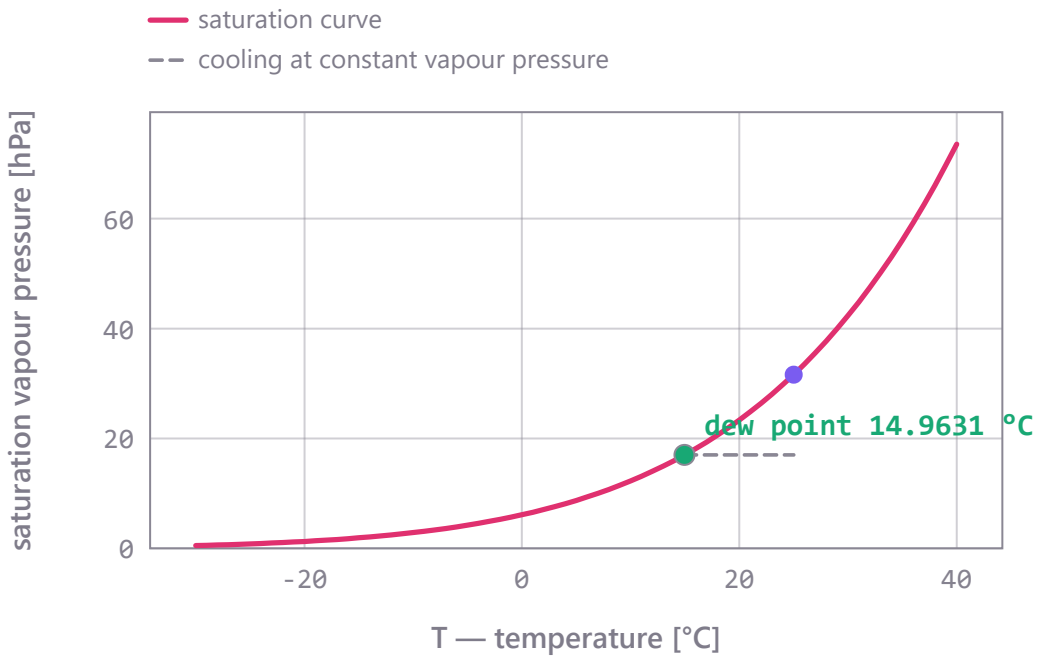
The equilibrium vapour pressure over a plane surface of liquid water (Ambaum 2020, latent heat varying with temperature).

$$e_s = e_{s0}\left(\frac{T_0}{T}\right)^{(c_{pl}-c_{pv})/R_v} \exp\left(\frac{L_0}{R_v T_0} - \frac{L(T)}{R_v T}\right)$$

input	meaning	condition
`T`	temperature	

output	meaning
`es`	saturation vapour pressure (Pa)

- **Method:** Ambaum (2020) Eq. 13 with L(T) = L0 - (c_pl - c_pv)$(T - T_0)$
- **Assumes:** -45 °$C \leq T \leq$ 60 °C (liquid water, possibly supercooled)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Saturation vapour pressure of water against temperature, computed by the package at 71 temperatures from −30 to 40 °C. Air holding e = 1700 Pa of vapour saturates when cooled to its dew point; at 25 °C saturation is 3162.35 Pa. Marked: Td = 14.9631 °C (288.113 K), e = 1700 Pa.

```
earth_science_ai999x.solve('saturation_vapor_pressure', T=298.15)
# -> es = 3162.35 Pa   (SOLVED)
```

```
earth_science_ai999x.solve('saturation_vapor_pressure', T=268.15)
# -> es = 421.541 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Saturation vapour pressure with temperature = 25 °C.	SOLVED	es = 3162.35 Pa
Saturation vapour pressure with temperature = -5 °C.	SOLVED	es = 421.541 Pa
Saturation vapour pressure with temperature = 25 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Saturation vapour pressure with temperature = 90 °C.	INVALID_INPUT	Saturation vapour pressure over water: temperature must lie between -45 and 60 °C.

Relative humidity from temperature and dewpoint — `relative_humidity_from_dewpoint`

Relative humidity as the ratio of the vapour pressure at the dewpoint to the saturation vapour pressure.

$$RH = \frac{e_s(T_d)}{e_s(T)}$$

input	meaning	condition
`T`	air temperature	
`Td`	dewpoint temperature	

output	meaning
`RH`	relative humidity (fraction)
`RH_percent`	relative humidity (%)

- **Method:** e_s(Td)/e_s(T), Ambaum (2020) e_s
- **Assumes:** $Td \leq T$; -45 °$C \leq T, Td \leq 60$ °C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('relative_humidity_from_dewpoint', T=298.15, Td=288.15)
# -> RH = 0.538557, RH_percent = 53.8557   (SOLVED)
```

```
earth_science_ai999x.solve('relative_humidity_from_dewpoint', T=278.15, Td=275.15)
# -> RH = 0.809202, RH_percent = 80.9202   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Relative humidity from dewpoint with temperature = 25 °C and dewpoint = 15 °C.	SOLVED	RH = 0.538557, RH_percent = 53.8557
Relative humidity from dewpoint with temperature = 5 °C and dewpoint = 2 °C.	SOLVED	RH = 0.809202, RH_percent = 80.9202
Relative humidity from dewpoint with temperature = 25 °C.	NEEDS_INPUT	relative_humidity_from_dewpoint needs Td (dewpoint temperature). Example: solve('relative_humidity_from_dewpoint', T='29
Relative humidity from dewpoint with temperature = 25 °C and dewpoint = 15 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Relative humidity from dewpoint with temperature = 10 °C and dewpoint = 15 °C.	INVALID_INPUT	Relative humidity from temperature and dewpoint: the dewpoint cannot exceed the temperature.

Dewpoint from vapour pressure — `dewpoint_from_vapor_pressure`

The dewpoint at which the given vapour pressure would saturate (inversion of Bolton's 1980 formula).

$$T_d = \frac{243.5\ \ln(e/6.112)}{17.67 - \ln(e/6.112)}\ \text{°C}$$

input	meaning	condition
`e`	water vapour partial pressure	

output	meaning
`Td`	dewpoint (K)

- **Method:** $Td = 243.5\,v/(17.67 - v)$ °C with $v = ln(e/6.112$ hPa)
- **Assumes:** 0.1 hPa <= e <= 200 hPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('dewpoint_from_vapor_pressure', e=1700.0)
# -> Td = 288.113 K   (SOLVED)
```

```
earth_science_ai999x.solve('dewpoint_from_vapor_pressure', e=450.0)
# -> Td = 269.003 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dewpoint from vapour pressure with vapour pressure = 17 hPa.	SOLVED	Td = 288.113 K
Dewpoint from vapour pressure with vapour pressure = 4.5 hPa.	SOLVED	Td = 269.003 K
Dewpoint from vapour pressure with vapour pressure = 17 hPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Dewpoint from temperature and relative humidity — `dewpoint_from_relative_humidity`

The dewpoint of air at temperature T and relative humidity RH: Bolton's inversion applied to RH e_s(T).

$$T_d = T_d^{\mathrm{Bolton}}(RH\, e_s(T))$$

input	meaning	condition
`T`	air temperature	
`RH`	relative humidity (fraction, or % in a question)	

output	meaning
`Td`	dewpoint (K)

- **Method:** Td = Bolton inversion of RH e_s(T), e_s from Ambaum (2020) (MetPy's convention)
- **Assumes:** $0.01 \le RH \le 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('dewpoint_from_relative_humidity', T=303.15, RH=0.6)
# -> Td = 294.506 K   (SOLVED)
```

```
earth_science_ai999x.solve('dewpoint_from_relative_humidity', T=273.15, RH=0.85)
# -> Td = 270.921 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dewpoint from relative humidity with temperature = 30 °C and relative humidity = 60 %.	SOLVED	Td = 294.506 K
Dewpoint from relative humidity with temperature = 0 °C and relative humidity = 85 %.	SOLVED	Td = 270.921 K
Dewpoint from relative humidity with temperature = 30 °C.	NEEDS_INPUT	dewpoint_from_relative_humidity needs RH (relative humidity (fraction, or % in a question)). Example: solve('dewpoint_fr
Dewpoint from relative humidity with temperature = 30 °C and relative humidity = 60 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Dewpoint from relative humidity with temperature = 20 °C and relative humidity = 150 %.	INVALID_INPUT	Dewpoint from temperature and relative humidity: relative humidity must be a fraction between 0.01 and 1 (1% to 100%).

Water-vapour mixing ratio from vapour pressure — `mixing_ratio`

Mass of water vapour per mass of dry air from the vapour partial pressure and the total pressure.

$$w = \frac{\varepsilon\, e}{p - e}$$

input	meaning	condition
`e`	water vapour partial pressure	
`p`	total air pressure	

output	meaning
`w`	mixing ratio (kg/kg)
`w_g_per_kg`	mixing ratio (g/kg)

- **Method:** w = epsilon e/(p - e)
- **Assumes:** $0 < e < p$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('mixing_ratio', e=2000.0, p=100000.0)
# -> w = 0.012693, w_g_per_kg = 12.693   (SOLVED)
```

```
earth_science_ai999x.solve('mixing_ratio', e=800.0, p=85000.0)
# -> w = 0.00590933, w_g_per_kg = 5.90933   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mixing ratio from vapour pressure with vapour pressure = 20 hPa and pressure = 1000 hPa.	SOLVED	w = 0.012693, w_g_per_kg = 12.693
Mixing ratio from vapour pressure with vapour pressure = 8 hPa and pressure = 850 hPa.	SOLVED	w = 0.00590933, w_g_per_kg = 5.90933
Mixing ratio from vapour pressure with vapour pressure = 20 hPa.	NEEDS_INPUT	mixing_ratio needs p (total air pressure). Example: solve('mixing_ratio', e='2000 Pa', p='100000 Pa').
Mixing ratio from vapour pressure with vapour pressure = 20 hPa and pressure = 1000 hPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Saturation mixing ratio — `saturation_mixing_ratio`

The mixing ratio of air saturated with respect to liquid water at temperature T and pressure p.

$$w_s = \frac{\varepsilon\, e_s(T)}{p - e_s(T)}$$

input	meaning	condition
`p`	total air pressure	
`T`	air temperature	

output	meaning
`ws`	saturation mixing ratio (kg/kg)
`ws_g_per_kg`	saturation mixing ratio (g/kg)

- **Method:** w_s = epsilon e_s/(p - e_s), Ambaum (2020) e_s
- **Assumes:** e_s(T) < p
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('saturation_mixing_ratio', p=100000.0, T=298.15)
# -> ws = 0.0203107, ws_g_per_kg = 20.3107   (SOLVED)
```

```
earth_science_ai999x.solve('saturation_mixing_ratio', p=50000.0, T=253.14999999999998)
# -> ws = 0.00156496, ws_g_per_kg = 1.56496   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Saturation mixing ratio with pressure = 1000 hPa and temperature = 25 °C.	SOLVED	ws = 0.0203107, ws_g_per_kg = 20.3107
Saturation mixing ratio with pressure = 500 hPa and temperature = -20 °C.	SOLVED	ws = 0.00156496, ws_g_per_kg = 1.56496

question	outcome	result
Saturation mixing ratio with pressure = 1000 hPa.	NEEDS_INPUT	saturation_mixing_ratio needs T (air temperature). Example: solve('saturation_mixing_ratio', p='100000 Pa', T='298.15 K'
Saturation mixing ratio with pressure = 1000 hPa and temperature = 25 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Virtual temperature — `virtual_temperature`

The temperature dry air would need to have the density of the moist air at the same pressure.

$$T_v = T\,\frac{w+\varepsilon}{\varepsilon(1+w)}$$

input	meaning	condition
`T`	air temperature	
`w`	water-vapour mixing ratio (kg/kg)	

output	meaning
`Tv`	virtual temperature (K)

- **Method:** $Tv = T$ (w + epsilon)/(epsilon $(1 + w)$)
- **Assumes:** $0 \le w \le 0.06$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('virtual_temperature', T=303.15, w=0.02)
# -> Tv = 306.763 K   (SOLVED)
```

```
earth_science_ai999x.solve('virtual_temperature', T=283.15, w=0.006)
# -> Tv = 284.176 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Virtual temperature with temperature = 30 °C and mixing ratio = 0.02.	SOLVED	Tv = 306.763 K
Virtual temperature with temperature = 10 °C and mixing ratio = 0.006.	SOLVED	Tv = 284.176 K
Virtual temperature with temperature = 30 °C.	NEEDS_INPUT	virtual_temperature needs w (water-vapour mixing ratio (kg/kg)). Example: solve('virtual_temperature', T='303.15 K', w=0
Virtual temperature with temperature = 30 °C and mixing ratio = 0.02 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Density of moist air — `moist_air_density`

Density of moist air from the ideal-gas law with the virtual temperature.

$$\rho = \frac{p}{R_d T_v}$$

input	meaning	condition
`p`	air pressure	
`T`	air temperature	
`w`	water-vapour mixing ratio (kg/kg)	

output	meaning
`rho`	air density (kg/m^3)
`Tv`	virtual temperature (K)

- **Method:** rho = p/(R_d T_v)
- **Assumes:** $0 \leq w \leq 0.06$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('moist_air_density', p=100000.0, T=298.15, w=0.015)
# -> rho = 1.15805 kg/m^3, Tv = 300.828 K   (SOLVED)
```

```
earth_science_ai999x.solve('moist_air_density', p=70000.0, T=273.15, w=0.004)
# -> rho = 0.89062 kg/m^3, Tv = 273.811 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Density of moist air with pressure = 1000 hPa, temperature = 25 °C and mixing ratio = 0.015.	SOLVED	rho = 1.15805 kg/m^3, Tv = 300.828 K
Density of moist air with pressure = 700 hPa, temperature = 0 °C and mixing ratio = 0.004.	SOLVED	rho = 0.89062 kg/m^3, Tv = 273.811 K
Density of moist air with pressure = 1000 hPa and temperature = 25 °C.	NEEDS_INPUT	moist_air_density needs w (water-vapour mixing ratio (kg/kg)). Example: solve('moist_air_density', p='100000 Pa', T='298
Density of moist air with pressure = 1000 hPa, temperature = 25 °C and mixing ratio = 0.015 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Specific humidity from dewpoint and pressure — `specific_humidity_from_dewpoint`

Specific humidity (mass of vapour per mass of moist air) from the dewpoint and the pressure.

$$q = \frac{w_s(p, T_d)}{1 + w_s(p, T_d)}$$

input	meaning	condition
`p`	air pressure	
`Td`	dewpoint temperature	

output	meaning
`q`	specific humidity (kg/kg)
`w`	mixing ratio (kg/kg)

- **Method:** $q = w_s$(p, Td)$/(1 + w_s$(p, Td))
- **Assumes:** e_s(Td) < p
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('specific_humidity_from_dewpoint', p=100000.0, Td=288.15)
# -> q = 0.0106612, w = 0.0107761   (SOLVED)
```

```
earth_science_ai999x.solve('specific_humidity_from_dewpoint', p=85000.0, Td=278.15)
# -> q = 0.00640331, w = 0.00644458   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Specific humidity from dewpoint with pressure = 1000 hPa and dewpoint = 15 °C.	SOLVED	q = 0.0106612, w = 0.0107761
Specific humidity from dewpoint with pressure = 850 hPa and dewpoint = 5 °C.	SOLVED	q = 0.00640331, w = 0.00644458
Specific humidity from dewpoint with pressure = 1000 hPa.	NEEDS_INPUT	specific_humidity_from_dewpoint needs Td (dewpoint temperature). Example: solve('specific_humidity_from_dewpoint', p='10
Specific humidity from dewpoint with pressure = 1000 hPa and dewpoint = 15 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Moist static energy — `moist_static_energy`

Moist static energy c_pd T + g z + L_v q (and the dry static energy c_pd T + g z) per unit mass.

$$h = c_{pd}T + gz + L_v q$$

input	meaning	condition
`z`	height above sea level	

input	meaning	condition
`T`	air temperature	
`q`	specific humidity (kg/kg)	

output	meaning
`h`	moist static energy (J/kg)
`s`	dry static energy (J/kg)

- **Method:** $h = c_p d$ T + g z + L_v q (MetPy's $L_v = 2.50084e6$ J/kg)
- **Assumes:** $0 \leq q \leq 0.06$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('moist_static_energy', z=1500.0, T=288.15, q=0.008)
# -> h = 324211 J/kg, s = 304205 J/kg   (SOLVED)
```

```
earth_science_ai999x.solve('moist_static_energy', z=0.0, T=301.15, q=0.018)
# -> h = 347570 J/kg, s = 302555 J/kg   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moist static energy with height = 1500 m, temperature = 15 °C and specific humidity = 0.008.	SOLVED	h = 324211 J/kg, s = 304205 J/kg
Moist static energy with height = 0 m, temperature = 28 °C and specific humidity = 0.018.	SOLVED	h = 347570 J/kg, s = 302555 J/kg
Moist static energy with height = 1500 m and temperature = 15 °C.	NEEDS_INPUT	moist_static_energy needs q (specific humidity (kg/kg)). Example: solve('moist_static_energy', z='1500 m', T='288.15 K',
Moist static energy with height = 1500 m, temperature = 15 °C and specific humidity = 0.008 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Lifting condensation level (Romps 2017) — `lifting_condensation_level`

Pressure and temperature at which a parcel lifted dry-adiabatically from (p, T, Td) becomes saturated, from Romps' exact solution with the Lambert W function.

$$T_{LCL} = \frac{c}{W_{-1}\left(RH^{1/a}\, c\, e^{c}\right)}\, T, \quad p_{LCL} = p\left(\frac{T_{LCL}}{T}\right)^{c_{pm}/R_m}$$

input	meaning	condition
`p`	starting pressure	
`T`	starting temperature	

input	meaning	condition
`Td`	starting dewpoint	

output	meaning
`p_lcl`	LCL pressure (Pa)
`T_lcl`	LCL temperature (K)

- **Method:** Romps (2017) Eq. 22: T_LCL = c/W_{-1}(RH^{$1/a$} c e^c) T, p_LCL = p (T_LCL/T)^{c_pm/R_m}
- **Assumes:** $Td \leq T$; 100 hPa <= p <= 1100 hPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('lifting_condensation_level', p=100000.0, T=303.15, Td=293.15)
# -> p_lcl = 86421.3 Pa, T_lcl = 290.812 K   (SOLVED)
```

```
earth_science_ai999x.solve('lifting_condensation_level', p=95000.0, T=288.15, Td=283.15)
# -> p_lcl = 88111.3 Pa, T_lcl = 282.031 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lifting condensation level with pressure = 1000 hPa, temperature = 30 °C and dewpoint = 20 °C.	SOLVED	p_lcl = 86421.3 Pa, T_lcl = 290.812 K
Lifting condensation level with pressure = 950 hPa, temperature = 15 °C and dewpoint = 10 °C.	SOLVED	p_lcl = 88111.3 Pa, T_lcl = 282.031 K
Lifting condensation level with pressure = 1000 hPa and temperature = 30 °C.	NEEDS_INPUT	lifting_condensation_level needs Td (starting dewpoint). Example: solve('lifting_condensation_level', p='100000 Pa', T='
Lifting condensation level with pressure = 1000 hPa, temperature = 30 °C and dewpoint = 20 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Lifting condensation level with pressure = 1000 hPa, temperature = 10 °C and dewpoint = 12 °C.	INVALID_INPUT	Lifting condensation level (Romps 2017): the dewpoint cannot exceed the temperature.

Equivalent potential temperature (Bolton 1980) — `equivalent_potential_temperature`

The potential temperature a parcel would have after all its vapour condensed and the latent heat warmed it.

$$\theta_e = T\left(\frac{p_0}{p-e}\right)^{\kappa}\left(\frac{T}{T_L}\right)^{0.28r}\exp\left[r(1+0.448r)\left(\frac{3036}{T_L}-1.78\right)\right]$$

input	meaning	condition
`p`	air pressure	
`T`	air temperature	

input	meaning	condition
`Td`	dewpoint	

output	meaning
`theta_e`	equivalent potential temperature (K)

- **Method:** Bolton (1980): T_L from T and Td, theta_L with the dry-air partial pressure, then the exponential factor
- **Assumes:** $Td \leq T$; 100 hPa <= p <= 1100 hPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('equivalent_potential_temperature', p=85000.0, T=293.15, Td=288.15)
# -> theta_e = 345.676 K   (SOLVED)
```

```
earth_science_ai999x.solve('equivalent_potential_temperature', p=100000.0, T=303.15, Td=297.15)
# -> theta_e = 360.128 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equivalent potential temperature with pressure = 850 hPa, temperature = 20 °C and dewpoint = 15 °C.	SOLVED	theta_e = 345.676 K
Equivalent potential temperature with pressure = 1000 hPa, temperature = 30 °C and dewpoint = 24 °C.	SOLVED	theta_e = 360.128 K
Equivalent potential temperature with pressure = 850 hPa and temperature = 20 °C.	NEEDS_INPUT	equivalent_potential_temperature needs Td (dewpoint). Example: solve('equivalent_potential_temperature', p='85000 Pa', T
Equivalent potential temperature with pressure = 850 hPa, temperature = 20 °C and dewpoint = 15 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Saturated (moist) adiabatic lapse rate — `moist_adiabatic_lapse_rate`

The rate at which a saturated parcel cools as it rises, with condensation releasing latent heat.

$$\Gamma_m = g \frac{1 + \frac{L_v r_s}{R_d T}}{c_{pd} + \frac{L_v^2 r_s}{R_v T^2}}$$

input	meaning	condition
`T`	air temperature	
`p`	air pressure	

output	meaning
`lapse_rate`	moist adiabatic lapse rate (K/m)
`rs`	saturation mixing ratio (kg/kg)

- **Method:** $Gamma_m = g$ (1 + L_v r_s/(R_d T)) / (c_pd + L_v^2 r_s/(R_v T^2)), r_s from Ambaum (2020) e_s
- **Assumes:** -40 °$C \le T \le 50$ °C; constant latent heat $L_v = 2.50084e6$ J/kg in the lapse-rate formula
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('moist_adiabatic_lapse_rate', T=293.15, p=100000.0)
# -> lapse_rate = 0.00422189 K/m, rs = 0.0148683   (SOLVED)
```

```
earth_science_ai999x.solve('moist_adiabatic_lapse_rate', T=273.15, p=70000.0)
# -> lapse_rate = 0.0057625 K/m, rs = 0.00547439   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moist adiabatic lapse rate with temperature = 20 °C and pressure = 1000 hPa.	SOLVED	lapse_rate = 0.00422189 K/m, rs = 0.0148683
Moist adiabatic lapse rate with temperature = 0 °C and pressure = 700 hPa.	SOLVED	lapse_rate = 0.0057625 K/m, rs = 0.00547439
Moist adiabatic lapse rate with temperature = 20 °C.	NEEDS_INPUT	moist_adiabatic_lapse_rate needs p (air pressure). Example: solve('moist_adiabatic_lapse_rate', T='293.15 K', p='100000
Moist adiabatic lapse rate with temperature = 20 °C and pressure = 1000 hPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Wind chill temperature index — `wind_chill`

The wind chill temperature index of the 2001 North American formula (wind measured at 10 m).

$$WCT = 13.12 + 0.6215\,T - 11.37\,V^{0.16} + 0.3965\,T\,V^{0.16}$$

input	meaning	condition
`T`	air temperature	
`V`	wind speed at 10 m	

output	meaning
`WCT`	wind chill temperature (K)

- **Method:** WCT = 13.12 + 0.6215 $T - 11.37\ V^{0.16} + 0.3965$ T $V^{0.16}$ (T in °C, V in km/h)
- **Assumes:** $T \le 10$ °C; $V > 3$ mph (4.8 km/h)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('wind_chill', T=263.15, V=8.333333333333334)
# -> WCT = 253.63 K   (SOLVED)
```

```
earth_science_ai999x.solve('wind_chill', T=273.15, V=5.555555555555555)
# -> WCT = 267.908 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wind chill with temperature = -10 °C and wind speed = 30 km/h.	SOLVED	WCT = 253.63 K
Wind chill with temperature = 0 °C and wind speed = 20 km/h.	SOLVED	WCT = 267.908 K
Wind chill with temperature = -10 °C.	NEEDS_INPUT	wind_chill needs V (wind speed at 10 m). Example: solve('wind_chill', T='263.15 K', V='8.33333 m/s').
Wind chill with temperature = -10 °C and wind speed = 30 km/h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Wind chill with temperature = 20 °C and wind speed = 30 km/h.	INVALID_INPUT	Wind chill temperature index: the wind chill index is defined for temperatures of 10 °C (50 °F) or below.

Heat index (NWS) — `heat_index`

The apparent temperature of the US National Weather Service: Rothfusz's regression with the NWS adjustments for low and high humidity.

$$HI = -42.379 + 2.049T + 10.143RH - 0.2248\,T\,RH - \ldots \ (^\circ\mathrm{F},\ RH \text{ in } \%)$$

input	meaning	condition
`T`	air temperature	
`RH`	relative humidity (fraction, or % in a question)	

output	meaning
`HI`	heat index (K)

- **Method:** Rothfusz (1990) regression with the NWS adjustments (as MetPy), computed in °F
- **Assumes:** 80 °$F \le T \le 130$ °F; $0 \le RH \le 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('heat_index', T=305.15, RH=0.7000000000000001)
# -> HI = 313.559 K   (SOLVED)
```

```
earth_science_ai999x.solve('heat_index', T=311.15, RH=0.1)
# -> HI = 307.877 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Heat index with temperature = 32 °C and relative humidity = 70 %.	SOLVED	HI = 313.559 K
Heat index with temperature = 38 °C and relative humidity = 10 %.	SOLVED	HI = 307.877 K
Heat index with temperature = 32 °C.	NEEDS_INPUT	heat_index needs RH (relative humidity (fraction, or % in a question)). Example: solve('heat_index', T='305.15 K', RH=0.
Heat index with temperature = 32 °C and relative humidity = 70 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Heat index with temperature = 20 °C and relative humidity = 50 %.	INVALID_INPUT	Heat index (NWS): the heat index is defined for temperatures from 80 °F (26.7 °C) to 130 °F.

Coriolis parameter — `coriolis_parameter`

The Coriolis parameter $f = 2$ Omega sin(latitude).

$$f = 2\Omega \sin \varphi$$

input	meaning	condition
`lat`	latitude (degrees, north positive)	

output	meaning
`f`	Coriolis parameter $(1/s)$

- **Method:** $f = 2$ Omega sin(phi), $Omega = 7.292115e - 5$ rad/s
- **Assumes:** -90 <= latitude <= 90 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('coriolis_parameter', lat=45.0)
# -> f = 0.000103126 1/s   (SOLVED)
```

```
earth_science_ai999x.solve('coriolis_parameter', lat=-30.0)
# -> f = -7.29211e-05 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coriolis parameter with latitude = 45 °.	SOLVED	f = 0.000103126 1/s
Coriolis parameter with latitude = -30 °.	SOLVED	f = -7.29211e-05 1/s
Coriolis parameter with latitude = 45 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Coriolis parameter with latitude = 100 °.	INVALID_INPUT	Coriolis parameter: latitude must be between -90 and 90 degrees.

Period of inertial oscillations — `inertial_period`

The period 2 pi/|f| of free inertial oscillations of the wind or an ocean current at a latitude.

$$T_i = \frac{2\pi}{|f|}$$

input	meaning	condition
`lat`	latitude (degrees, north positive)	

output	meaning
`T_inertial`	inertial period (s)
`T_hours`	inertial period (hours)
`f`	Coriolis parameter $(1/s)$

- **Method:** $T = 2$ pi/|f| with $f = 2$ Omega sin(phi)
- **Assumes:** |latitude| >= 0.5 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('inertial_period', lat=45.0)
# -> T_inertial = 60927.2 s, T_hours = 16.9242, f = 0.000103126 1/s   (SOLVED)
```

```
earth_science_ai999x.solve('inertial_period', lat=30.0)
# -> T_inertial = 86164.1 s, T_hours = 23.9345, f = 7.29211e-05 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inertial period with latitude = 45 °.	SOLVED	T_inertial = 60927.2 s, T_hours = 16.9242, f = 0.000103126 1/s
Inertial period with latitude = 30 °.	SOLVED	T_inertial = 86164.1 s, T_hours = 23.9345, f = 7.29211e-05 1/s
Inertial period with latitude = 45 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Inertial period with latitude = 0 °.	INVALID_INPUT	Period of inertial oscillations: inertial oscillations need |latitude| >= 0.5 degrees (f = 0 at the equator).

Wind components from speed and direction — `wind_components`

The eastward (u) and northward (v) wind components from the speed and the direction the wind blows FROM.

$$u = -V\sin\delta, \quad v = -V\cos\delta$$

input	meaning	condition
`speed`	wind speed	

input	meaning	condition
`direction`	direction the wind blows from (degrees clockwise from north)	

output	meaning
u	eastward component (m/s)
v	northward component (m/s)

- **Method:** $u = -V$ sin(dir), $v = -V$ cos(dir)
- **Assumes:** 0 <= direction <= 360; meteorological convention: the direction the wind comes from
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('wind_components', speed=10.0, direction=225.0)
# -> u = 7.07107 m/s, v = 7.07107 m/s   (SOLVED)
```

```
earth_science_ai999x.solve('wind_components', speed=15.0, direction=300.0)
# -> u = 12.9904 m/s, v = -7.5 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wind components with wind speed = 10 m/s and wind direction = 225 °.	SOLVED	u = 7.07107 m/s, v = 7.07107 m/s
Wind components with wind speed = 15 m/s and wind direction = 300 °.	SOLVED	u = 12.9904 m/s, v = -7.5 m/s
Wind components with wind speed = 10 m/s.	NEEDS_INPUT	wind_components needs direction (direction the wind blows from (degrees clockwise from north)). Example: solve('wind_com
Wind components with wind speed = 10 m/s and wind direction = 225 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Wind speed and direction from components — `wind_speed_direction`

Wind speed and the direction the wind blows from (degrees clockwise from north, (0, 360]) from u and v.

$$V = \sqrt{u^2 + v^2}, \quad \delta = 90^\circ - \operatorname{atan2}(-v, -u)$$

input	meaning	condition
u	eastward wind component	
v	northward wind component	

output	meaning
`speed`	wind speed (m/s)
`direction`	direction the wind comes from (degrees)

- **Method:** V = hypot(u, v); direction = 90 - atan2(-v, -u) in degrees
- **Assumes:** calm $(u = v = 0)$ gives direction 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('wind_speed_direction', u=5.0, v=-3.0)
# -> speed = 5.83095 m/s, direction = 300.964   (SOLVED)
```

```
earth_science_ai999x.solve('wind_speed_direction', u=-8.0, v=6.0)
# -> speed = 10 m/s, direction = 126.87   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wind speed and direction with u component = 5 m/s and v component = -3 m/s.	SOLVED	speed = 5.83095 m/s, direction = 300.964
Wind speed and direction with u component = -8 m/s and v component = 6 m/s.	SOLVED	speed = 10 m/s, direction = 126.87
Wind speed and direction with u component = 5 m/s.	NEEDS_INPUT	wind_speed_direction needs v (northward wind component). Example: solve('wind_speed_direction', u='5 m/s', v='-3 m/s').
Wind speed and direction with u component = 5 m/s and v component = -3 m/s and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Geostrophic wind from a height gradient — `geostrophic_wind`

Geostrophic wind speed from the change of geopotential height of a pressure surface over a horizontal distance.

$$V_g = \frac{g}{|f|}\left|\frac{\Delta Z}{\Delta n}\right|$$

input	meaning	condition
`lat`	latitude (degrees, north positive)	
`dz`	change of geopotential height across the distance	
`dn`	horizontal distance over which the height changes	

output	meaning
`Vg`	geostrophic wind speed (m/s)
`f`	Coriolis parameter $(1/s)$

- **Method:** V_g = (g/|f|) |dZ/dn|
- **Assumes:** 5 <= |latitude| <= 85 degrees; $dn > 0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('geostrophic_wind', lat=45.0, dz=60.0, dn=500000.0)
# -> Vg = 11.4113 m/s, f = 0.000103126 1/s   (SOLVED)
```

```
earth_science_ai999x.solve('geostrophic_wind', lat=30.0, dz=40.0, dn=300000.0)
# -> Vg = 17.9311 m/s, f = 7.29211e-05 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geostrophic wind with latitude = 45 °, height difference = 60 m and distance = 500 km.	SOLVED	Vg = 11.4113 m/s, f = 0.000103126 1/s
Geostrophic wind with latitude = 30 °, height difference = 40 m and distance = 300 km.	SOLVED	Vg = 17.9311 m/s, f = 7.29211e-05 1/s
Geostrophic wind with latitude = 45 ° and height difference = 60 m.	NEEDS_INPUT	geostrophic_wind needs dn (horizontal distance over which the height changes). Example: solve('geostrophic_wind', lat=45
Geostrophic wind with latitude = 45 °, height difference = 60 m and distance = 500 km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Geostrophic wind with latitude = 0 °, height difference = 60 m and distance = 500 km.	INVALID_INPUT	Geostrophic wind from a height gradient: geostrophic balance is used here for 5 to 85 degrees latitude.

Brunt-Väisälä (buoyancy) frequency — `brunt_vaisala_frequency`

The buoyancy frequency of a stably stratified atmosphere from the potential temperature and its vertical gradient.

$$N = \sqrt{\frac{g}{\theta}\frac{\partial \theta}{\partial z}}$$

input	meaning	condition
`theta`	potential temperature at the level	
`dtheta_dz`	vertical gradient of potential temperature	

output	meaning
`N`	Brunt-Väisälä frequency $(1/s)$
`period`	buoyancy period 2 pi/N (s)

- **Method:** $N = sqrt$((g/theta) d theta/dz)
- **Assumes:** d $\theta/dz > 0$ (stable stratification)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('brunt_vaisala_frequency', theta=300.0, dtheta_dz=0.003)
# -> N = 0.00990285 1/s, period = 634.482 s   (SOLVED)
```

```
earth_science_ai999x.solve('brunt_vaisala_frequency', theta=350.0, dtheta_dz=0.015)
# -> N = 0.0205009 1/s, period = 306.484 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Brunt vaisala frequency with potential temperature = 300 K and potential temperature gradient = 3 K/km.	SOLVED	N = 0.00990285 1/s, period = 634.482 s
Brunt vaisala frequency with potential temperature = 350 K and potential temperature gradient = 15 K/km.	SOLVED	N = 0.0205009 1/s, period = 306.484 s
Brunt vaisala frequency with potential temperature = 300 K.	NEEDS_INPUT	brunt_vaisala_frequency needs dtheta_dz (vertical gradient of potential temperature). Example: solve('brunt_vaisala_freq
Brunt vaisala frequency with potential temperature = 300 K and potential temperature gradient = 3 K/km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Brunt vaisala frequency with potential temperature = 300 K and potential temperature gradient = -2 K/km.	INVALID_INPUT	Brunt-Väisälä (buoyancy) frequency: the potential-temperature gradient must be positive (stable) and at most 0.1 K/m.

Pressure at height in an isothermal atmosphere — `isothermal_pressure`

The barometric formula: pressure at height z in an isothermal dry atmosphere, and its scale height R_d T/g.

$$p = p_0\, e^{-z/H}, \quad H = \frac{R_d T}{g}$$

input	meaning	condition
`p0`	pressure at $z = 0$	
`T`	temperature of the layer	
`z`	height above the reference level	

output	meaning
`p`	pressure at height z (Pa)
`H`	scale height (m)

- **Method:** $p = p_0$ exp(-z/H), $H = R_d$ T/g
- **Assumes:** isothermal dry air; constant gravity $g = 9.80665\ m/s^2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('isothermal_pressure', p0=101325.0, T=273.15, z=5000.0)
# -> p = 54215.4 Pa, H = 7995.29 m   (SOLVED)
```

```
earth_science_ai999x.solve('isothermal_pressure', p0=100000.0, T=253.14999999999998, z=2000.0)
# -> p = 76344.8 Pa, H = 7409.88 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Isothermal atmosphere with surface pressure = 1013.25 hPa, temperature = 0 °C and height = 5000 m.	SOLVED	p = 54215.4 Pa, H = 7995.29 m
Isothermal atmosphere with surface pressure = 1000 hPa, temperature = -20 °C and height = 2000 m.	SOLVED	p = 76344.8 Pa, H = 7409.88 m
Isothermal atmosphere with surface pressure = 1013.25 hPa and temperature = 0 °C.	NEEDS_INPUT	isothermal_pressure needs z (height above the reference level). Example: solve('isothermal_pressure', p0='101325 Pa', T=
Isothermal atmosphere with surface pressure = 1013.25 hPa, temperature = 0 °C and height = 5000 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Atmospheric scale height — `scale_height`

The height over which pressure falls by a factor e in an isothermal dry atmosphere.

$$H = \frac{R_d T}{g}$$

input	meaning	condition
`T`	temperature	

output	meaning
`H`	scale height (m)

- **Method:** $H = R_d$ T/g
- **Assumes:** dry air, constant gravity
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('scale_height', T=288.15)
# -> H = 8434.35 m   (SOLVED)
```

```
earth_science_ai999x.solve('scale_height', T=223.14999999999998)
# -> H = 6531.76 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Scale height with temperature = 15 °C.	SOLVED	H = 8434.35 m
Scale height with temperature = -50 °C.	SOLVED	H = 6531.76 m
Scale height with temperature = 15 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Thickness of a layer (hypsometric equation) — `hypsometric_thickness`

The geometric thickness between two pressure levels from the layer-mean virtual temperature.

$$\Delta z = \frac{R_d \overline{T_v}}{g} \ln \frac{p_1}{p_2}$$

input	meaning	condition
`p1`	pressure at the bottom of the layer	
`p2`	pressure at the top of the layer	
`Tv`	layer-mean virtual temperature	

output	meaning
`dz`	layer thickness (m)

- **Method:** dz = (R_d Tv/g) ln(p1/p2)
- **Assumes:** $p_1 > p_2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('hypsometric_thickness', p1=100000.0, p2=50000.0, Tv=263.15)
# -> dz = 5339.02 m   (SOLVED)
```

```
earth_science_ai999x.solve('hypsometric_thickness', p1=85000.0, p2=70000.0, Tv=278.15)
# -> dz = 1580.75 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hypsometric equation with bottom pressure = 1000 hPa, top pressure = 500 hPa and mean virtual temperature = -10 °C.	SOLVED	dz = 5339.02 m
Hypsometric equation with bottom pressure = 850 hPa, top pressure = 700 hPa and mean virtual temperature = 5 °C.	SOLVED	dz = 1580.75 m
Hypsometric equation with bottom pressure = 1000 hPa and top pressure =	NEEDS_INPUT	hypsometric_thickness needs Tv (layer-mean virtual temperature). Example: solve('hypsometric_thickness',

question	outcome	result
500 hPa.		p1='100000 Pa',
Hypsometric equation with bottom pressure = 1000 hPa, top pressure = 500 hPa and mean virtual temperature = -10 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Hypsometric equation with bottom pressure = 500 hPa, top pressure = 1000 hPa and mean virtual temperature = -10 °C.	INVALID_INPUT	Thickness of a layer (hypsometric equation): the bottom pressure p1 must exceed the top pressure p2.

Pressure at height with a constant lapse rate — `pressure_with_lapse_rate`

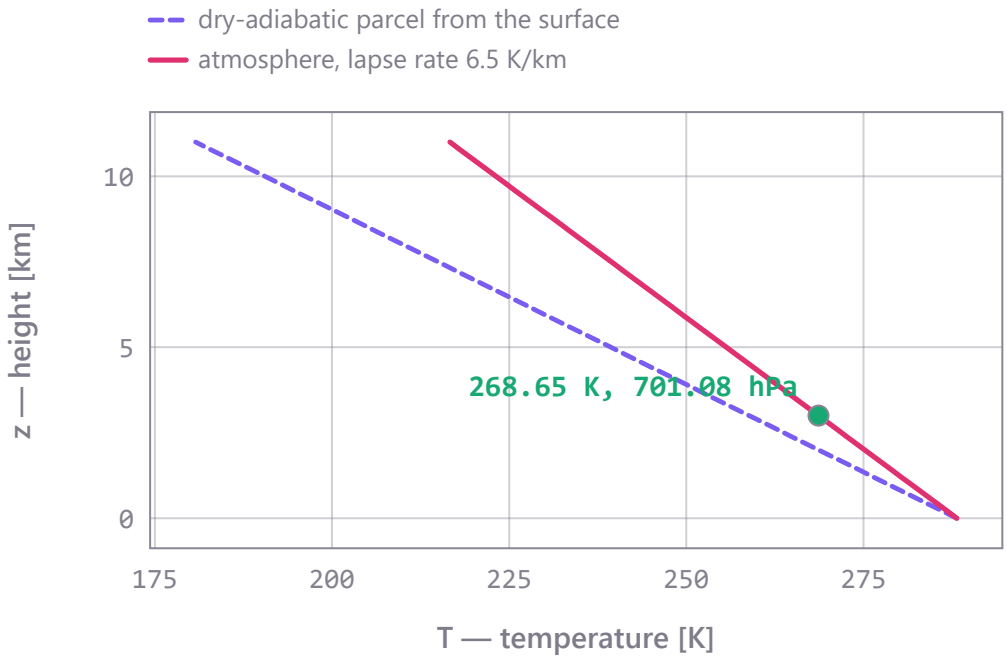
Pressure at height z in a dry atmosphere whose temperature falls linearly with height (troposphere model).

$$p = p_0 \left(\frac{T_0 - \gamma z}{T_0} \right)^{g/(R_d \gamma)}$$

input	meaning	condition
`p0`	pressure at $z = 0$	
`T0`	temperature at $z = 0$	
`gamma`	temperature lapse rate (positive when temperature falls with height)	
`z`	height above the reference level	

output	meaning
`p`	pressure at height z (Pa)
`T`	temperature at height z (K)

- **Method:** $p = p_0$ (T/T0)^(g/(R_d gamma)), $T = T_0 - \gamma$ z
- **Assumes:** $0 < \gamma \leq 9.8$ K/km; dry air, constant gravity
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Temperature against height in an atmosphere with a constant lapse rate of 6.5 K/km from 288.15 K and 101325 Pa at the surface, and a dry parcel lifted adiabatically from the same surface temperature, both computed by the package every 200 m to 11 km. Marked: z = 3 km, T = 268.65 K, p = 70108 Pa.

```
earth_science_ai999x.solve('pressure_with_lapse_rate', p0=101325.0, T0=288.15,
gamma=0.006500000000000001, z=3000.0)
# -> p = 70108 Pa, T = 268.65 K   (SOLVED)
```

```
earth_science_ai999x.solve('pressure_with_lapse_rate', p0=100000.0, T0=298.15, gamma=0.008,
z=1500.0)
# -> p = 83909.3 Pa, T = 286.15 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Constant lapse rate with surface pressure = 1013.25 hPa, surface temperature = 15 °C, lapse rate = 6.5 K/km and height = 3000 m.	SOLVED	p = 70108 Pa, T = 268.65 K
Constant lapse rate with surface pressure = 1000 hPa, surface temperature = 25 °C, lapse rate = 8 K/km and height = 1500 m.	SOLVED	p = 83909.3 Pa, T = 286.15 K
Constant lapse rate with surface pressure = 1013.25 hPa, surface temperature = 15 °C and lapse rate = 6.5 K/km.	NEEDS_INPUT	pressure_with_lapse_rate needs z (height above the reference level). Example: solve('pressure_with_lapse_rate', p0='1013
Constant lapse rate with surface pressure = 1013.25 hPa, surface temperature = 15 °C, lapse rate = 6.5 K/km and height = 3000 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Pressure at a height in the standard atmosphere — `standard_atmosphere_pressure`

Pressure at a height in the tropospheric standard atmosphere (T0 = 288 K, 6.5 K/km, 1013.25 hPa; as MetPy).

$$p = p_0 \left(1 - \frac{\gamma z}{T_0}\right)^{g/(R_d \gamma)}$$

input	meaning	condition
z	height above sea level	

output	meaning
p	standard-atmosphere pressure (Pa)

- **Method:** $p = p_0$ (1 - gamma z/T0)^(g/(R_d gamma))
- **Assumes:** -500 $m \leq z \leq 11$ km (troposphere)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('standard_atmosphere_pressure', z=1500.0)
# -> p = 84547.6 Pa   (SOLVED)
```

```
earth_science_ai999x.solve('standard_atmosphere_pressure', z=9000.0)
# -> p = 30720.3 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Standard atmosphere pressure with height = 1500 m.	SOLVED	p = 84547.6 Pa
Standard atmosphere pressure with height = 9000 m.	SOLVED	p = 30720.3 Pa
Standard atmosphere pressure with height = 1500 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Standard atmosphere pressure with height = 20000 m.	INVALID_INPUT	Pressure at a height in the standard atmosphere: the tropospheric standard atmosphere covers -500 m to 11 km.

Height of a pressure level in the standard atmosphere — `standard_atmosphere_height`

The height of a pressure level in the tropospheric standard atmosphere (pressure altitude; as MetPy).

$$z = \frac{T_0}{\gamma}\left[1 - \left(\frac{p}{p_0}\right)^{R_d \gamma / g}\right]$$

input	meaning	condition
p	pressure	

output	meaning
z	pressure altitude (m)

- **Method:** $z = (T_0/\gamma)$(1 - (p/p0)^(R_d gamma/g))

- **Assumes:** 226 hPa <= p <= 1075 hPa
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('standard_atmosphere_height', p=50000.0)
# -> z = 5571.43 m   (SOLVED)
```

```
earth_science_ai999x.solve('standard_atmosphere_height', p=85000.0)
# -> z = 1456.51 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Standard atmosphere height with pressure = 500 hPa.	SOLVED	z = 5571.43 m
Standard atmosphere height with pressure = 850 hPa.	SOLVED	z = 1456.51 m
Standard atmosphere height with pressure = 500 hPa and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Standard atmosphere height with pressure = 100 hPa.	INVALID_INPUT	Height of a pressure level in the standard atmosphere: the tropospheric standard atmosphere covers about 1075 hPa to 226

Geopotential and geopotential height — `geopotential_height`

Geopotential and geopotential height of a geometric height above sea level, with gravity falling off as $1/(R_e + z)^2$ (as MetPy).

$$\Phi = \frac{g\,R_e\,z}{R_e + z}, \quad Z = \Phi / g$$

input	meaning	condition
`z`	geometric height above sea level	

output	meaning
`phi`	geopotential (m^2/s^2)
`Z`	geopotential height (m)

- **Method:** $Phi = g$ R_e z/(R_e + z)
- **Assumes:** -1 $km \le z \le 200$ km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('geopotential_height', z=10000.0)
# -> phi = 97912.8 m^2/s^2, Z = 9984.33 m   (SOLVED)
```

```
earth_science_ai999x.solve('geopotential_height', z=50000.0)
# -> phi = 486514 m^2/s^2, Z = 49610.7 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geopotential height with height = 10 km.	SOLVED	phi = 97912.8 m^2/s^2, Z = 9984.33 m
Geopotential height with height = 50 km.	SOLVED	phi = 486514 m^2/s^2, Z = 49610.7 m
Geopotential height with height = 10 km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Phase speed of a barotropic Rossby wave — `rossby_wave_speed`

Zonal phase speed of a barotropic Rossby wave in a uniform westerly flow on a beta-plane, the beta parameter, and the stationary wavelength 2 pi sqrt(U/beta).

$$c = U - \frac{\beta}{k^2}, \quad \beta = \frac{2\Omega\cos\varphi}{a}, \quad L_s = 2\pi\sqrt{U/\beta}$$

input	meaning	condition
`U`	zonal mean (westerly) wind speed	
`lat`	latitude (degrees, north positive)	
`Lx`	zonal wavelength	

output	meaning
`c`	zonal phase speed (m/s; negative = westward)
`beta`	$\beta = df/dy$ (1/(m s))
`Ls`	stationary wavelength (m)

- **Method:** $c = U - \beta/k^2$, $\beta = 2$ Omega cos(phi)/a, $L_s = 2$ pi sqrt(U/beta)
- **Assumes:** $U > 0$; 5 <= |latitude| <= 85; no meridional wavenumber $(l = 0)$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('rossby_wave_speed', U=15.0, lat=45.0, Lx=6000000.0)
# -> c = 0.239432 m/s, beta = 1.61868e-11 1/(m*s), Ls = 6.04847e+06 m   (SOLVED)
```

```
earth_science_ai999x.solve('rossby_wave_speed', U=10.0, lat=60.0, Lx=4000000.0)
# -> c = 5.3612 m/s, beta = 1.14458e-11 1/(m*s), Ls = 5.87296e+06 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rossby wave with mean wind = 15 m/s, latitude = 45 ° and wavelength	SOLVED	c = 0.239432 m/s, beta = 1.61868e-11 1/(m*s), Ls = 6.04847e+06 m

question	outcome	result
= 6000 km.		
Rossby wave with mean wind = 10 m/s, latitude = 60 ° and wavelength = 4000 km.	SOLVED	c = 5.3612 m/s, beta = 1.14458e-11 1/(m*s), Ls = 5.87296e+06 m
Rossby wave with mean wind = 15 m/s and latitude = 45 °.	NEEDS_INPUT	rossby_wave_speed needs Lx (zonal wavelength). Example: solve('rossby_wave_speed', U='15 m/s', lat=45, Lx='6e+06 m').
Rossby wave with mean wind = 15 m/s, latitude = 45 ° and wavelength = 6000 km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Radiant flux emitted by a black or grey body (Stefan-Boltzmann) — `blackbody_flux`

Power emitted per unit area by a surface of temperature T and (grey) emissivity eps.

$$F = \varepsilon \sigma T^4$$

input	meaning	condition
`T`	surface temperature	
`eps`	emissivity (0 < eps <= 1; default 1, a black body)	

output	meaning
`F`	emitted flux (W/m^2)

- **Method:** F = eps sigma T^4 (sigma = 5.670374419e-8 W/(m^2 K^4))
- **Assumes:** 0 < eps <= 1; grey body: emissivity independent of wavelength
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('blackbody_flux', T=288.0, eps=1.0)
# -> F = 390.105 W/m^2   (SOLVED)
```

```
earth_science_ai999x.solve('blackbody_flux', T=255.0, eps=0.95)
# -> F = 227.77 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stefan boltzmann with temperature = 288 K and emissivity = 1.	SOLVED	F = 390.105 W/m^2
Stefan boltzmann with temperature = 255 K and emissivity = 0.95.	SOLVED	F = 227.77 W/m^2
Stefan boltzmann with temperature = 288 K and emissivity = 1 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Wavelength of peak blackbody emission (Wien) — `wien_peak_wavelength`

The wavelength at which a black body of temperature T emits most strongly per unit wavelength.

$$\lambda_{\max} = \frac{b}{T}, \quad b = 2.897771955 \times 10^{-3}\ \mathrm{m\,K}$$

input	meaning	condition
`T`	temperature of the body	

output	meaning
`lambda_max`	peak wavelength (m)
`lambda_um`	peak wavelength (micrometres)

- **Method:** lambda_max = b/T, $b = 2.897771955e-3$ m K
- **Assumes:** 1 $K \le T \le 1e5$ K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('wien_peak_wavelength', T=5772.0)
# -> lambda_max = 5.02039e-07 m, lambda_um = 0.502039   (SOLVED)
```

```
earth_science_ai999x.solve('wien_peak_wavelength', T=288.0)
# -> lambda_max = 1.00617e-05 m, lambda_um = 10.0617   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wien peak wavelength with temperature = 5772 K.	SOLVED	lambda_max = 5.02039e-07 m, lambda_um = 0.502039
Wien peak wavelength with temperature = 288 K.	SOLVED	lambda_max = 1.00617e-05 m, lambda_um = 10.0617
Wien peak wavelength with temperature = 5772 K and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Planetary equilibrium (effective radiating) temperature — `planetary_equilibrium_temperature`

The temperature at which a planet radiating as a black body balances the sunlight it absorbs.

$$T_e = \left(\frac{S(1-\alpha)}{4\sigma} \right)^{1/4}$$

input	meaning	condition
`S`	solar flux at the planet's distance (solar constant)	
`albedo`	planetary (Bond) albedo, a fraction	

output	meaning
`Te`	equilibrium temperature (K)
`absorbed`	absorbed flux per unit surface area, S(1 - albedo)/4 (W/m^2)

- **Method:** T_e = (S (1 - albedo)/(4 sigma))^$(1/4)$
- **Assumes:** 0 <= albedo < 1; fast rotation: absorbed sunlight spread over the whole sphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('planetary_equilibrium_temperature', S=1361.0, albedo=0.3)
# -> Te = 254.578 K, absorbed = 238.175 W/m^2   (SOLVED)
```

```
earth_science_ai999x.solve('planetary_equilibrium_temperature', S=589.0, albedo=0.25)
# -> Te = 210.076 K, absorbed = 110.438 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equilibrium temperature with solar constant = 1361 W/m^2 and albedo = 0.3.	SOLVED	Te = 254.578 K, absorbed = 238.175 W/m^2
Equilibrium temperature with solar constant = 589 W/m^2 and albedo = 0.25.	SOLVED	Te = 210.076 K, absorbed = 110.438 W/m^2
Equilibrium temperature with solar constant = 1361 W/m^2.	NEEDS_INPUT	planetary_equilibrium_temperature needs albedo (planetary (Bond) albedo, a fraction). Example: solve('planetary_equilibr
Equilibrium temperature with solar constant = 1361 W/m^2 and albedo = 0.3 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Surface temperature in the single-layer greenhouse model — `greenhouse_single_layer`

Surface and atmosphere temperatures when one isothermal layer, transparent to sunlight, absorbs and emits a fraction eps of thermal radiation.

$$T_s = \left[\frac{S(1-\alpha)}{4\sigma(1-\varepsilon/2)}\right]^{1/4}, \quad T_a = 2^{-1/4}T_s$$

input	meaning	condition
`S`	solar constant	
`albedo`	planetary (Bond) albedo, a fraction	
`eps`	long-wave emissivity (absorptivity) of the atmospheric layer	

output	meaning
`Ts`	surface temperature (K)
`Ta`	atmosphere temperature (K)

- **Method:** T_s = [S(1 - albedo)/(4 sigma (1 - eps/2))]^$(1/4)$, $T_a = T_s/2^{1/4}$
- **Assumes:** the layer is transparent to sunlight; 0 < eps <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('greenhouse_single_layer', S=1361.0, albedo=0.3, eps=0.78)
# -> Ts = 288.064 K, Ta = 242.232 K   (SOLVED)
```

```
earth_science_ai999x.solve('greenhouse_single_layer', S=1361.0, albedo=0.3, eps=1.0)
# -> Ts = 302.746 K, Ta = 254.578 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Single layer greenhouse with solar constant = 1361 W/m^2, albedo = 0.3 and emissivity = 0.78.	SOLVED	Ts = 288.064 K, Ta = 242.232 K
Single layer greenhouse with solar constant = 1361 W/m^2, albedo = 0.3 and emissivity = 1.	SOLVED	Ts = 302.746 K, Ta = 254.578 K
Single layer greenhouse with solar constant = 1361 W/m^2 and albedo = 0.3.	NEEDS_INPUT	greenhouse_single_layer needs eps (long-wave emissivity (absorptivity) of the atmospheric layer). Example: solve('greenh
Single layer greenhouse with solar constant = 1361 W/m^2, albedo = 0.3 and emissivity = 0.78 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Surface temperature with N fully absorbing layers — `greenhouse_n_layers`

Surface and layer temperatures of the N-layer grey-atmosphere model (each layer absorbs all thermal radiation).

$$T_s = (N+1)^{1/4}\,T_e, \quad T_e = \left(\frac{S(1-\alpha)}{4\sigma}\right)^{1/4}$$

input	meaning	condition
`S`	solar constant	
`albedo`	planetary (Bond) albedo, a fraction	
`N`	number of absorbing layers (whole number)	

output	meaning
`Ts`	surface temperature (K)
`T_top`	temperature of the top layer (K)
`T_bottom_layer`	temperature of the lowest layer (K)

- **Method:** $T_s = (N+1)^{1/4}$ T_e, layer i from the top at $i^{1/4}$ T_e
- **Assumes:** $1 \le N \le 100$ whole layers; layers transparent to sunlight, opaque to thermal radiation

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('greenhouse_n_layers', S=1361.0, albedo=0.3, N=2)
# -> Ts = 335.044 K, T_top = 254.578 K, T_bottom_layer = 302.746 K   (SOLVED)
```

```
earth_science_ai999x.solve('greenhouse_n_layers', S=2601.0, albedo=0.75, N=10)
# -> Ts = 421.406 K, T_top = 231.394 K, T_bottom_layer = 411.483 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Layer greenhouse with solar constant = 1361 W/m^2, albedo = 0.3 and layers = 2.	SOLVED	Ts = 335.044 K, T_top = 254.578 K, T_bottom_layer = 302.746 K
Layer greenhouse with solar constant = 2601 W/m^2, albedo = 0.75 and layers = 10.	SOLVED	Ts = 421.406 K, T_top = 231.394 K, T_bottom_layer = 411.483 K
Layer greenhouse with solar constant = 1361 W/m^2 and albedo = 0.3.	NEEDS_INPUT	greenhouse_n_layers needs N (number of absorbing layers (whole number)). Example: solve('greenhouse_n_layers', S='1361 W
Layer greenhouse with solar constant = 1361 W/m^2, albedo = 0.3 and layers = 2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Daily mean insolation at the top of the atmosphere — `daily_mean_insolation`

The 24-hour mean solar flux on a horizontal surface at the top of the atmosphere for a latitude and solar declination (polar day and polar night included).

$$\bar{Q} = \frac{S_0}{\pi}\left(h_0 \sin\varphi \sin\delta + \cos\varphi \cos\delta \sin h_0\right)$$

input	meaning	condition
`lat`	latitude (degrees, north positive)	
`decl`	solar declination (degrees)	
`S0`	solar constant (at the actual Earth-Sun distance)	

output	meaning
`Q`	daily mean insolation (W/m^2)
`day_length_h`	length of the day (hours)

- **Method:** $Q = (S_0/pi)$(h0 sin phi sin delta + cos phi cos delta sin h0)
- **Assumes:** |latitude| < 89.9; |declination| <= 23.5
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

0	0	75	272	533
24	106	217	346	492
151	241	332	419	499
286	352	407	447	471
397	424	433	424	397
471	447	407	352	286
499	419	332	241	151
492	346	217	106	24
533	272	75	0	0

W/m² · rows 80° N → 80° S · columns Dec. → June

Daily mean solar radiation at the top of the atmosphere (solar constant 1361 W/m², rounded to whole W/m²) for latitudes 80° N to 80° S (rows) and solar declinations −23.44°, −11.72°, 0°, 11.72° and 23.44° (columns), each cell computed by the package: the summer pole receives the most, the winter pole none. At latitude 45° and δ = 23.44°: 499.319 W/m², day length 15.4259 h.

```
earth_science_ai999x.solve('daily_mean_insolation', lat=45.0, decl=23.44, S0=1361.0)
# -> Q = 499.319 W/m^2, day_length_h = 15.4259   (SOLVED)
```

```
earth_science_ai999x.solve('daily_mean_insolation', lat=80.0, decl=20.0, S0=1361.0)
# -> Q = 458.418 W/m^2, day_length_h = 24   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Daily mean insolation with latitude = 45 °, declination = 23.44 ° and solar constant = 1361 W/m^2.	SOLVED	Q = 499.319 W/m^2, day_length_h = 15.4259

question	outcome	result
Daily mean insolation with latitude = 80 °, declination = 20 ° and solar constant = 1361 W/m^2.	SOLVED	Q = 458.418 W/m^2, day_length_h = 24
Daily mean insolation with latitude = 45 ° and declination = 23.44 °.	NEEDS_INPUT	daily_mean_insolation needs S0 (solar constant (at the actual Earth-Sun distance)). Example: solve('daily_mean_insolatio
Daily mean insolation with latitude = 45 °, declination = 23.44 ° and solar constant = 1361 W/m^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Warming of a one-box energy-balance model — `energy_balance_response`

Temperature response after t years to a constant radiative forcing F in a one-box model C $dT/dt = F - T/\lambda$, the equilibrium warming lambda F and the e-folding time lambda C.

$$\Delta T(t) = \lambda F\left(1 - e^{-t/\tau}\right), \quad \tau = \lambda C$$

input	meaning	condition
`F`	radiative forcing applied at $t = 0$	
`lam`	climate sensitivity parameter	
`C`	effective heat capacity per unit area	
`t`	time since the forcing began (years)	

output	meaning
`dT`	warming after t (K)
`dT_eq`	equilibrium warming (K)
`tau_years`	e-folding time (years)

- **Method:** dT(t) = lambda F (1 - exp(-t/(lambda C)))
- **Assumes:** $\lambda > 0$, $C > 0$; the forcing is switched on at $t = 0$ and then held constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('energy_balance_response', F=3.7, lam=0.8, C=420000000.0, t=10.0)
# -> dT = 1.80283 K, dT_eq = 2.96 K, tau_years = 10.6472   (SOLVED)
```

```
earth_science_ai999x.solve('energy_balance_response', F=1.5, lam=0.5, C=800000000.0, t=30.0)
# -> dT = 0.679666 K, dT_eq = 0.75 K, tau_years = 12.6752   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Energy balance model with forcing = 3.7 W/m^2, sensitivity = 0.8 K/(W/m^2), heat	SOLVED	dT = 1.80283 K, dT_eq = 2.96 K, tau_years = 10.6472

question	outcome	result
capacity = 4.2e+08 J/(m^2*K) and time = 10 yr.		
Energy balance model with forcing = 1.5 W/m^2, sensitivity = 0.5 K/(W/m^2), heat capacity = 8e+08 J/(m^2*K) and time = 30 yr.	SOLVED	dT = 0.679666 K, dT_eq = 0.75 K, tau_years = 12.6752
Energy balance model with forcing = 3.7 W/m^2, sensitivity = 0.8 K/(W/m^2) and heat capacity = 4.2e+08 J/(m^2*K).	NEEDS_INPUT	energy_balance_response needs t (time since the forcing began (years)). Example: solve('energy_balance_response', F='3.7
Energy balance model with forcing = 3.7 W/m^2, sensitivity = 0.8 K/(W/m^2), heat capacity = 4.2e+08 J/(m^2*K) and time = 10 yr and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Density of pure water (SMOW, Bigg 1967) — `pure_water_density`

Density of standard mean ocean water (no salt) at atmospheric pressure, UNESCO 1983 polynomial.

$$\rho_w = \sum_{i=0}^{5} a_i \, T_{68}^{\,i}$$

input	meaning	condition
`T`	water temperature	

output	meaning
`rho`	density (kg/m^3)

- **Method:** Bigg (1967) polynomial in T68 (UNESCO 1983 Eq. 14)
- **Assumes:** 0 °$C \le T \le 40$ °C; atmospheric pressure
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('pure_water_density', T=277.15)
# -> rho = 999.975 kg/m^3   (SOLVED)
```

```
earth_science_ai999x.solve('pure_water_density', T=298.15)
# -> rho = 997.046 kg/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Density of pure water with temperature = 4 °C.	SOLVED	rho = 999.975 kg/m^3
Density of pure water with temperature = 25 °C.	SOLVED	rho = 997.046 kg/m^3
Density of pure water with temperature = 4 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Density of seawater at the surface (UNESCO 1981) — `seawater_density_surface`

Density of seawater at atmospheric pressure from practical salinity and temperature (EOS-80), and sigma-t.

$$\rho(S,T,0) = \rho_w + B(T)\,S + C(T)\,S^{3/2} + d\,S^2$$

input	meaning	condition
`S`	practical salinity (PSS-78)	
`T`	water temperature	

output	meaning
`rho`	density (kg/m^3)
`sigma_t`	$\sigma - t = \rho - 1000\ (kg/m^3)$

- **Method:** UNESCO (1981) one-atmosphere equation of state (EOS-80)
- **Assumes:** $0 \le S \le 42$; -2 °$C \le T \le 40$ °C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seawater_density_surface', S=35.0, T=288.15)
# -> rho = 1025.97 kg/m^3, sigma_t = 25.972   (SOLVED)
```

```
earth_science_ai999x.solve('seawater_density_surface', S=30.0, T=275.15)
# -> rho = 1023.97 kg/m^3, sigma_t = 23.9679   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Density of seawater at the surface with salinity = 35 and temperature = 15 °C.	SOLVED	rho = 1025.97 kg/m^3, sigma_t = 25.972
Density of seawater at the surface with salinity = 30 and temperature = 2 °C.	SOLVED	rho = 1023.97 kg/m^3, sigma_t = 23.9679
Density of seawater at the surface with salinity = 35.	NEEDS_INPUT	seawater_density_surface needs T (water temperature). Example: solve('seawater_density_surface', S=35, T='288.15 K').
Density of seawater at the surface with salinity = 35 and temperature = 15 °C and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

In-situ density of seawater (EOS-80) — `seawater_density`

In-situ density of seawater from practical salinity, temperature and sea pressure with the secant bulk modulus.

$$\rho(S,T,p) = \frac{\rho(S,T,0)}{1 - p/K(S,T,p)}$$

input	meaning	condition
`S`	practical salinity (PSS-78)	
`T`	in-situ temperature	
`p`	sea pressure (0 at the surface)	

output	meaning
`rho`	in-situ density (kg/m^3)
`sigma`	$\rho - 1000$ (kg/m^3)

- **Method:** $\rho = \rho$(S,T,$0)/(1 - P/K$(S,T,P)), P in bar (UNESCO 1983)
- **Assumes:** $0 \le S \le 42$; -2 °$C \le T \le 40$ °C; $0 \le p \le 10$,000 dbar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seawater_density', S=35.0, T=275.15, p=40000000.0)
# -> rho = 1046.02 kg/m^3, sigma = 46.0168   (SOLVED)
```

```
earth_science_ai999x.solve('seawater_density', S=34.5, T=283.15, p=10000000.0)
# -> rho = 1031.04 kg/m^3, sigma = 31.0429   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
In situ density of seawater with salinity = 35, temperature = 2 °C and pressure = 4000 dbar.	SOLVED	rho = 1046.02 kg/m^3, sigma = 46.0168
In situ density of seawater with salinity = 34.5, temperature = 10 °C and pressure = 1000 dbar.	SOLVED	rho = 1031.04 kg/m^3, sigma = 31.0429
In situ density of seawater with salinity = 35 and temperature = 2 °C.	NEEDS_INPUT	seawater_density needs p (sea pressure (0 at the surface)). Example: solve('seawater_density', S=35, T='275.15 K', p='4e
In situ density of seawater with salinity = 35, temperature = 2 °C and pressure = 4000 dbar and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Freezing point of seawater (UNESCO 1983) — `seawater_freezing_point`

The freezing temperature of seawater from practical salinity and sea pressure (Millero 1978).

$$t_f = -0.0575S + 1.710523 \times 10^{-3} S^{3/2} - 2.154996 \times 10^{-4} S^2 - 7.53 \times 10^{-4} p$$

input	meaning	condition
`S`	practical salinity (PSS-78)	
`p`	sea pressure (0 at the surface; default 0)	

output	meaning
`Tf`	freezing temperature (K)
`Tf_C`	freezing temperature (°C, ITS-90)

- **Method:** $t_f = -0.0575\,S + 1.710523e-3\,S^{1.5} - 2.154996e-4\,S^2 - 7.53e-4$ p (p in dbar)
- **Assumes:** $4 \le S \le 40$; $0 \le p \le 500$ dbar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seawater_freezing_point', S=35.0, p=0.0)
# -> Tf = 271.228 K, Tf_C = -1.92184   (SOLVED)
```

```
earth_science_ai999x.solve('seawater_freezing_point', S=34.0, p=3000000.0)
# -> Tf = 271.06 K, Tf_C = -2.0904   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Freezing point of seawater with salinity = 35 and pressure = 0 dbar.	SOLVED	Tf = 271.228 K, Tf_C = -1.92184
Freezing point of seawater with salinity = 34 and pressure = 300 dbar.	SOLVED	Tf = 271.06 K, Tf_C = -2.0904
Freezing point of seawater with salinity = 35 and pressure = 0 dbar and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Freezing point of seawater with salinity = 50 and pressure = 0 dbar.	INVALID_INPUT	Freezing point of seawater (UNESCO 1983): practical salinity must be between 0 and 42.

Speed of sound in seawater (Chen and Millero 1977) — `sound_speed_seawater`

Speed of sound in seawater from practical salinity, temperature and sea pressure (UNESCO 1983 algorithm).

$$c(S,T,P) = C_w(T,P) + A(T,P)\,S + B(T,P)\,S^{3/2} + D(P)\,S^2$$

input	meaning	condition
`S`	practical salinity (PSS-78)	
`T`	water temperature	
`p`	sea pressure (0 at the surface)	

output	meaning
`c`	speed of sound (m/s)

- **Method:** Chen and Millero (1977) polynomial, P in bar (UNESCO 1983 Eq. 33)
- **Assumes:** $0 \le S \le 40$; 0 °$C \le T \le 40$ °C; $0 \le p \le 10$,000 dbar

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('sound_speed_seawater', S=35.0, T=283.15, p=10000000.0)
# -> c = 1506.35 m/s   (SOLVED)
```

```
earth_science_ai999x.solve('sound_speed_seawater', S=35.0, T=275.15, p=50000000.0)
# -> c = 1542.66 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Speed of sound in seawater with salinity = 35, temperature = 10 °C and pressure = 1000 dbar.	SOLVED	c = 1506.35 m/s
Speed of sound in seawater with salinity = 35, temperature = 2 °C and pressure = 5000 dbar.	SOLVED	c = 1542.66 m/s
Speed of sound in seawater with salinity = 35 and temperature = 10 °C.	NEEDS_INPUT	sound_speed_seawater needs p (sea pressure (0 at the surface)). Example: solve('sound_speed_seawater', S=35, T='283.15 K
Speed of sound in seawater with salinity = 35, temperature = 10 °C and pressure = 1000 dbar and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Sea pressure at a depth (Saunders 1981) — `ocean_pressure_from_depth`

Sea pressure at a depth in the ocean, allowing for latitude (Saunders 1981).

$$p = \frac{(1 - c_1) - \sqrt{(1 - c_1)^2 - 8.84 \times 10^{-6} z}}{4.42 \times 10^{-6}}$$

input	meaning	condition
`depth`	depth below the sea surface	
`lat`	latitude (degrees, north positive)	

output	meaning
`p`	sea pressure (Pa)
`p_dbar`	sea pressure (dbar)

- **Method:** Saunders (1981): p = [$(1 - c_1) - sqrt((1 - c_1)^2 - 8.84e-6$ z)]/$4.42e-6$
- **Assumes:** 0 <= depth <= 11,000 m
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('ocean_pressure_from_depth', depth=1000.0, lat=30.0)
# -> p = 1.00955e+07 Pa, p_dbar = 1009.55   (SOLVED)
```

```
earth_science_ai999x.solve('ocean_pressure_from_depth', depth=5000.0, lat=60.0)
# -> p = 5.10802e+07 Pa, p_dbar = 5108.02   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pressure at depth in the ocean with depth = 1000 m and latitude = 30 °.	SOLVED	p = 1.00955e+07 Pa, p_dbar = 1009.55
Pressure at depth in the ocean with depth = 5000 m and latitude = 60 °.	SOLVED	p = 5.10802e+07 Pa, p_dbar = 5108.02
Pressure at depth in the ocean with depth = 1000 m.	NEEDS_INPUT	ocean_pressure_from_depth needs lat (latitude (degrees, north positive)). Example: solve('ocean_pressure_from_depth', de
Pressure at depth in the ocean with depth = 1000 m and latitude = 30 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Depth from sea pressure (Fofonoff and Millard 1983) — `ocean_depth_from_pressure`

Depth in the ocean from sea pressure and latitude (UNESCO 1983 standard algorithm).

$$z = \frac{\sum c_i p^i}{g(\varphi) + \frac{1}{2}\gamma' p}$$

input	meaning	condition
`p`	sea pressure (0 at the surface)	
`lat`	latitude (degrees, north positive)	

output	meaning
`depth`	depth (m)

- **Method:** z = (9.72659 $p - 2.2512e-5\,p^2 + 2.279e-10\,p^3 - 1.82e-15\,p^4$)/($g$(phi) + 1.092e-6 p)
- **Assumes:** $0 \le p \le 12$,000 dbar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('ocean_depth_from_pressure', p=10000000.0, lat=30.0)
# -> depth = 990.808 m   (SOLVED)
```

```
earth_science_ai999x.solve('ocean_depth_from_pressure', p=50000000.0, lat=45.0)
# -> depth = 4902.08 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Depth from pressure with pressure = 1000 dbar and latitude = 30 °.	SOLVED	depth = 990.808 m
Depth from pressure with pressure = 5000 dbar and latitude = 45 °.	SOLVED	depth = 4902.08 m
Depth from pressure with pressure = 1000 dbar.	NEEDS_INPUT	ocean_depth_from_pressure needs lat (latitude (degrees, north positive)). Example: solve('ocean_depth_from_pressure', p=
Depth from pressure with pressure = 1000 dbar and latitude = 30 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Adiabatic temperature gradient of seawater (Bryden 1973) — `seawater_adiabatic_gradient`

How fast the temperature of seawater rises under adiabatic compression, per decibar.

$$\Gamma(S,T,p) = \left(\frac{\partial T}{\partial p}\right)_\eta$$

input	meaning	condition
`S`	practical salinity (PSS-78)	
`T`	in-situ temperature	
`p`	sea pressure (0 at the surface)	

output	meaning
`gamma`	adiabatic gradient (K per dbar)
`gamma_per_km_dbar`	adiabatic gradient (K per 1000 dbar)

- **Method:** Bryden (1973) polynomial (UNESCO 1983 Eq. 30)
- **Assumes:** $0 \le S \le 42$; -2 °$C \le T \le 40$ °C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seawater_adiabatic_gradient', S=35.0, T=283.15, p=10000000.0)
# -> gamma = 0.000127387, gamma_per_km_dbar = 0.127387   (SOLVED)
```

```
earth_science_ai999x.solve('seawater_adiabatic_gradient', S=34.7, T=275.15, p=40000000.0)
# -> gamma = 0.000115046, gamma_per_km_dbar = 0.115046   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Adiabatic temperature gradient of seawater with salinity = 35, temperature = 10 °C and pressure = 1000 dbar.	SOLVED	gamma = 0.000127387, gamma_per_km_dbar = 0.127387

question	outcome	result
Adiabatic temperature gradient of seawater with salinity = 34.7, temperature = 2 °C and pressure = 4000 dbar.	SOLVED	gamma = 0.000115046, gamma_per_km_dbar = 0.115046
Adiabatic temperature gradient of seawater with salinity = 35 and temperature = 10 °C.	NEEDS_INPUT	seawater_adiabatic_gradient needs p (sea pressure (0 at the surface)). Example: solve('seawater_adiabatic_gradient', S=3
Adiabatic temperature gradient of seawater with salinity = 35, temperature = 10 °C and pressure = 1000 dbar and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Potential temperature of seawater (Fofonoff 1977) — `seawater_potential_temperature`

The temperature a seawater parcel would have if moved adiabatically to a reference pressure (default the surface), by a fourth-order Runge-Kutta integration of the adiabatic gradient.

$$\theta = T + \int_{p}^{p_r} \Gamma(S, \theta', p')\, dp'$$

input	meaning	condition
`S`	practical salinity (PSS-78)	
`T`	in-situ temperature	
`p`	sea pressure (0 at the surface)	
`pr`	reference sea pressure (default 0, the surface)	

output	meaning
`theta`	potential temperature (K)
`theta_C`	potential temperature (°C)
`dtheta`	theta - T (K)

- **Method:** Fofonoff (1977) fourth-order Runge-Kutta integration of the adiabatic gradient from p to p_r
- **Assumes:** $0 \le S \le 42$; -2 °$C \le T \le 40$ °C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seawater_potential_temperature', S=35.0, T=276.15, p=50000000.0,
pr=0.0)
# -> theta = 275.663 K, theta_C = 2.5132, dtheta = -0.486797 K   (SOLVED)
```

```
earth_science_ai999x.solve('seawater_potential_temperature', S=34.7, T=283.15, p=20000000.0,
pr=0.0)
# -> theta = 282.898 K, theta_C = 9.74815, dtheta = -0.251852 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Potential temperature of seawater with salinity = 35, temperature = 3 °C, pressure = 5000 dbar and reference pressure = 0 dbar.	SOLVED	theta = 275.663 K, theta_C = 2.5132, dtheta = -0.486797 K
Potential temperature of seawater with salinity = 34.7, temperature = 10 °C, pressure = 2000 dbar and reference pressure = 0 dbar.	SOLVED	theta = 282.898 K, theta_C = 9.74815, dtheta = -0.251852 K
Potential temperature of seawater with salinity = 35, temperature = 3 °C and reference pressure = 0 dbar.	NEEDS_INPUT	seawater_potential_temperature needs p (sea pressure (0 at the surface)). Example: solve('seawater_potential_temperature
Potential temperature of seawater with salinity = 35, temperature = 3 °C, pressure = 5000 dbar and reference pressure = 0 dbar and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Gravitational acceleration at a latitude and height — `gravity_at_latitude`

Normal gravity at sea level for a latitude (International Gravity Formula 1967, as UNESCO 1983) reduced for height with the inverse-square law.

$$g = \frac{9.780318\,(1 + 5.2788\times10^{-3}\sin^2\varphi + 2.36\times10^{-5}\sin^4\varphi)}{(1 + z/R)^2}$$

input	meaning	condition
`lat`	latitude (degrees, north positive)	
`z`	height above sea level (default 0)	

output	meaning
g	gravitational acceleration (m/s^2)

- **Method:** $g = 9.780318$ (1 + 5.2788e-3 $sin^2\ \phi + 2.36e-5\ sin^4$ phi)/(1 + z/R)^2
- **Assumes:** -90 <= latitude <= 90; spherical inverse-square reduction for height (R = 6371 km)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('gravity_at_latitude', lat=45.0, z=0.0)
# -> g = 9.80619 m/s^2   (SOLVED)
```

```
earth_science_ai999x.solve('gravity_at_latitude', lat=0.0, z=4000.0)
# -> g = 9.76805 m/s^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gravity at latitude with latitude = 45 ° and height = 0 m.	SOLVED	g = 9.80619 m/s^2

question	outcome	result
Gravity at latitude with latitude = 0 ° and height = 4000 m.	SOLVED	g = 9.76805 m/s^2
Gravity at latitude with latitude = 45 ° and height = 0 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Speed of a shallow-water (long) wave and tsunami travel time — `shallow_water_wave`

Speed sqrt(g h) of a wave much longer than the water depth (tsunami, tide), and its travel time over a distance.

$$c = \sqrt{gh}, \quad t = d/c$$

input	meaning	condition
`h`	water depth	
`distance`	distance travelled (optional)	

output	meaning
`c`	wave speed (m/s)
`travel_time`	travel time (s)
`travel_time_h`	travel time (hours)

- **Method:** $c = sqrt$(g h), $t = d/c$
- **Assumes:** wavelength much longer than the depth; standard gravity
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('shallow_water_wave', h=4000.0, distance=5000000.0)
# -> c = 198.057 m/s, travel_time = 25245.2 s, travel_time_h = 7.01257   (SOLVED)
```

```
earth_science_ai999x.solve('shallow_water_wave', h=10.0, distance=1000.0)
# -> c = 9.90285 m/s, travel_time = 100.981 s, travel_time_h = 0.0280503   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Shallow water wave with depth = 4000 m and distance = 5000 km.	SOLVED	c = 198.057 m/s, travel_time = 25245.2 s, travel_time_h = 7.01257
Shallow water wave with depth = 10 m and distance = 1 km.	SOLVED	c = 9.90285 m/s, travel_time = 100.981 s, travel_time_h = 0.0280503
Shallow water wave with depth = 4000 m and distance = 5000 km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Ekman layer depth — `ekman_depth`

Depth of frictional influence of the wind on the ocean (the current reverses direction at $D_E = pi$ delta).

$$D_E = \pi\sqrt{\frac{2A_z}{|f|}}$$

input	meaning	condition
`Az`	vertical eddy viscosity	
`lat`	latitude (degrees, north positive)	

output	meaning
`D_E`	Ekman depth pi delta (m)
`delta`	Ekman e-folding depth sqrt(2 Az/|f|) (m)

- **Method:** $D_E = pi$ sqrt(2 Az/|f|)
- **Assumes:** 5 <= |latitude| <= 89; constant eddy viscosity; deep ocean
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('ekman_depth', Az=0.05, lat=45.0)
# -> D_E = 97.8286 m, delta = 31.1398 m   (SOLVED)
```

```
earth_science_ai999x.solve('ekman_depth', Az=0.01, lat=-30.0)
# -> D_E = 52.0281 m, delta = 16.5611 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ekman depth with eddy viscosity = 0.05 m^2/s and latitude = 45 °.	SOLVED	D_E = 97.8286 m, delta = 31.1398 m
Ekman depth with eddy viscosity = 0.01 m^2/s and latitude = -30 °.	SOLVED	D_E = 52.0281 m, delta = 16.5611 m
Ekman depth with eddy viscosity = 0.05 m^2/s.	NEEDS_INPUT	ekman_depth needs lat (latitude (degrees, north positive)). Example: solve('ekman_depth', Az='0.05 m^2/s', lat=45).
Ekman depth with eddy viscosity = 0.05 m^2/s and latitude = 45 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Ekman transport — `ekman_transport`

Volume transport per unit width of the wind-driven Ekman layer, directed 90 degrees to the right of the wind in the northern hemisphere (left in the southern).

$$M_E = \frac{\tau}{\rho\,|f|}$$

input	meaning	condition
`tau`	wind stress on the sea surface	

input	meaning	condition
`lat`	latitude (degrees, north positive)	
`rho`	seawater density (default 1025 kg/m^3)	

output	meaning
`M`	Ekman volume transport per unit width (m^2/s)
`mass_transport`	Ekman mass transport per unit width (kg/(m s))

- **Method:** M = tau/(rho |f|)
- **Assumes:** 5 <= |latitude| <= 89; steady state, deep ocean
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('ekman_transport', tau=0.1, lat=45.0, rho=1025.0)
# -> M = 0.946036 m^2/s, mass_transport = 969.687 kg/(m*s)   (SOLVED)
```

```
earth_science_ai999x.solve('ekman_transport', tau=0.2, lat=-30.0, rho=1027.0)
# -> M = 2.67058 m^2/s, mass_transport = 2742.69 kg/(m*s)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ekman transport with wind stress = 0.1 Pa, latitude = 45 ° and density = 1025 kg/m^3.	SOLVED	M = 0.946036 m^2/s, mass_transport = 969.687 kg/(m*s)
Ekman transport with wind stress = 0.2 Pa, latitude = -30 ° and density = 1027 kg/m^3.	SOLVED	M = 2.67058 m^2/s, mass_transport = 2742.69 kg/(m*s)
Ekman transport with wind stress = 0.1 Pa and density = 1025 kg/m^3.	NEEDS_INPUT	ekman_transport needs lat (latitude (degrees, north positive)). Example: solve('ekman_transport', tau='0.1 Pa', lat=45,
Ekman transport with wind stress = 0.1 Pa, latitude = 45 ° and density = 1025 kg/m^3 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

P- and S-wave speeds from bulk and shear moduli — `seismic_velocities_from_moduli`

Body-wave speeds of an isotropic elastic rock from its bulk modulus, shear modulus and density.

$$V_P = \sqrt{\frac{K + \frac{4}{3}\mu}{\rho}}, \quad V_S = \sqrt{\frac{\mu}{\rho}}$$

input	meaning	condition
`K`	bulk modulus (incompressibility)	
`mu`	shear modulus (rigidity)	
`rho`	density of the rock	

output	meaning
`Vp`	P-wave speed (m/s)
`Vs`	S-wave speed (m/s)
`Vp_Vs`	Vp/Vs

- **Method:** $Vp = sqrt$((K + 4 mu/3)/rho), $Vs = \sqrt{\frac{\mu}{\rho}}$
- **Assumes:** isotropic, linear elastic; $K > 0$, $mu > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seismic_velocities_from_moduli', K=50000000000.0, mu=30000000000.0,
rho=2700.0)
# -> Vp = 5773.5 m/s, Vs = 3333.33 m/s, Vp_Vs = 1.73205   (SOLVED)
```

```
earth_science_ai999x.solve('seismic_velocities_from_moduli', K=130000000000.0, mu=70000000000.0,
rho=3300.0)
# -> Vp = 8226.59 m/s, Vs = 4605.66 m/s, Vp_Vs = 1.78619   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Seismic velocities from moduli with bulk modulus = 50 GPa, shear modulus = 30 GPa and density = 2700 kg/m^3.	SOLVED	Vp = 5773.5 m/s, Vs = 3333.33 m/s, Vp_Vs = 1.73205
Seismic velocities from moduli with bulk modulus = 130 GPa, shear modulus = 70 GPa and density = 3300 kg/m^3.	SOLVED	Vp = 8226.59 m/s, Vs = 4605.66 m/s, Vp_Vs = 1.78619
Seismic velocities from moduli with bulk modulus = 50 GPa and shear modulus = 30 GPa.	NEEDS_INPUT	seismic_velocities_from_moduli needs rho (density of the rock). Example: solve('seismic_velocities_from_moduli', K='5e+1
Seismic velocities from moduli with bulk modulus = 50 GPa, shear modulus = 30 GPa and density = 2700 kg/m^3 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

P- and S-wave speeds from Young's modulus and Poisson's ratio — `seismic_velocities_from_young`

Body-wave speeds of an isotropic elastic rock from Young's modulus, Poisson's ratio and density.

$$V_P = \sqrt{\frac{E(1-\nu)}{\rho(1+\nu)(1-2\nu)}}, \quad V_S = \sqrt{\frac{E}{2\rho(1+\nu)}}$$

input	meaning	condition
`E`	Young's modulus	

input	meaning	condition
`nu`	Poisson's ratio	
`rho`	density of the rock	

output	meaning
`Vp`	P-wave speed (m/s)
`Vs`	S-wave speed (m/s)
`Vp_Vs`	Vp/Vs

- **Method:** Lamé constants from E and nu, then $Vp = sqrt$((lambda + 2 mu)/rho), $Vs = \sqrt{\frac{\mu}{\rho}}$
- **Assumes:** isotropic, linear elastic; $-1 < \nu < 0.5$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seismic_velocities_from_young', E=70000000000.0, nu=0.25, rho=2700.0)
# -> Vp = 5577.73 m/s, Vs = 3220.31 m/s, Vp_Vs = 1.73205   (SOLVED)
```

```
earth_science_ai999x.solve('seismic_velocities_from_young', E=160000000000.0, nu=0.28, rho=3300.0)
# -> Vp = 7872.96 m/s, Vs = 4351.94 m/s, Vp_Vs = 1.80907   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Seismic velocities from young's modulus with young's modulus = 70 GPa, poisson's ratio = 0.25 and density = 2700 kg/m^3.	SOLVED	Vp = 5577.73 m/s, Vs = 3220.31 m/s, Vp_Vs = 1.73205
Seismic velocities from young's modulus with young's modulus = 160 GPa, poisson's ratio = 0.28 and density = 3300 kg/m^3.	SOLVED	Vp = 7872.96 m/s, Vs = 4351.94 m/s, Vp_Vs = 1.80907
Seismic velocities from young's modulus with young's modulus = 70 GPa and poisson's ratio = 0.25.	NEEDS_INPUT	seismic_velocities_from_young needs rho (density of the rock). Example: solve('seismic_velocities_from_young', E='7e+10
Seismic velocities from young's modulus with young's modulus = 70 GPa, poisson's ratio = 0.25 and density = 2700 kg/m^3 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Seismic velocities from young's modulus with young's modulus = 70 GPa, poisson's ratio = 0.6 and density = 2700 kg/m^3.	INVALID_INPUT	P- and S-wave speeds from Young's modulus and Poisson's ratio: Poisson's ratio must be between -1 and 0.5.

Elastic moduli and Poisson's ratio from Vp, Vs and density — `elastic_constants_from_velocities`

Shear modulus, Lamé's lambda, bulk modulus, Young's modulus and Poisson's ratio of an isotropic rock from its seismic velocities and density.

$$\mu = \rho V_S^2, \quad \lambda = \rho(V_P^2 - 2V_S^2), \quad \nu = \frac{V_P^2 - 2V_S^2}{2(V_P^2 - V_S^2)}$$

input	meaning	condition
`Vp`	P-wave speed	
`Vs`	S-wave speed	
`rho`	density of the rock	

output	meaning
`mu`	shear modulus (Pa)
`lam`	Lamé's first parameter (Pa)
`K`	bulk modulus (Pa)
`E`	Young's modulus (Pa)
`nu`	Poisson's ratio

- **Method:** $mu = \rho\, Vs^2$, $\lambda = \rho$ (Vp^2 - 2 Vs^2), $K = \lambda + 2\, mu/3$, nu = (Vp^2 - 2Vs^2)/(2(Vp^2 - Vs^2))
- **Assumes:** isotropic, linear elastic; $Vp > sqrt$(4/3) Vs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('elastic_constants_from_velocities', Vp=6000.0, Vs=3460.0, rho=2700.0)
# -> mu = 3.23233e+10 Pa, lam = 3.25534e+10 Pa, K = 5.41022e+10 Pa, E = 8.08656e+10 Pa, nu =
0.250886   (SOLVED)
```

```
earth_science_ai999x.solve('elastic_constants_from_velocities', Vp=8100.0, Vs=4500.0, rho=3350.0)
# -> mu = 6.78375e+10 Pa, lam = 8.41185e+10 Pa, K = 1.29344e+11 Pa, E = 1.73228e+11 Pa, nu =
0.276786   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Elastic constants from velocities with p wave speed = 6 km/s, s wave speed = 3.46 km/s and density = 2700 kg/m^3.	SOLVED	mu = 3.23233e+10 Pa, lam = 3.25534e+10 Pa, K = 5.41022e+10 Pa, E = 8.08656e+10 Pa, nu = 0.250886
Elastic constants from velocities with p wave speed = 8.1 km/s, s wave speed = 4.5 km/s and density = 3350 kg/m^3.	SOLVED	mu = 6.78375e+10 Pa, lam = 8.41185e+10 Pa, K = 1.29344e+11 Pa, E = 1.73228e+11 Pa, nu = 0.276786
Elastic constants from velocities with p wave speed = 6 km/s and s wave speed = 3.46 km/s.	NEEDS_INPUT	elastic_constants_from_velocities needs rho (density of the rock). Example: solve('elastic_constants_from_velocities', V
Elastic constants from velocities with p wave speed = 6 km/s, s wave speed = 3.46 km/s and density = 2700 kg/m^3 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

question	outcome	result
Elastic constants from velocities with p wave speed = 3 km/s, s wave speed = 3 km/s and density = 2700 kg/m^3.	INVALID_INPUT	Elastic moduli and Poisson's ratio from Vp, Vs and density: Vp must exceed sqrt(4/3) Vs (a positive bulk modulus).

Epicentral distance on a spherical Earth — `epicentral_distance`

Great-circle angular distance between an epicentre and a station, and its length on a sphere of radius 6371 km.

$$\Delta = \operatorname{atan2}\left(\sqrt{(\cos\varphi_2 \sin\Delta\lambda)^2 + (\cos\varphi_1 \sin\varphi_2 - \sin\varphi_1 \cos\varphi_2 \cos\Delta\lambda)^2},\ \sin\varphi_1 \sin\varphi_2 + \cos\varphi_1 \cos\varphi_2\right.$$

input	meaning	condition
`lat1`	latitude of the first point (epicentre)	
`lon1`	longitude of the first point (epicentre)	
`lat2`	latitude of the second point (station)	
`lon2`	longitude of the second point (station)	

output	meaning
`delta`	epicentral distance (degrees)
`distance_km`	great-circle distance (km)

- **Method:** great-circle angle by atan2 (Vincenty's formula on the sphere)
- **Assumes:** spherical Earth of radius 6371 km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('epicentral_distance', lat1=35.7, lon1=139.7, lat2=51.5, lon2=-0.1)
# -> delta = 85.9539, distance_km = 9557.64   (SOLVED)
```

```
earth_science_ai999x.solve('epicentral_distance', lat1=0.0, lon1=0.0, lat2=30.0, lon2=40.0)
# -> delta = 48.4392, distance_km = 5386.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Epicentral distance with epicentre latitude = 35.7 °, epicentre longitude = 139.7 °, station latitude = 51.5 ° and station longitude = -0.1 °.	SOLVED	delta = 85.9539, distance_km = 9557.64
Epicentral distance with epicentre latitude = 0 °, epicentre longitude = 0 °, station latitude = 30 ° and station longitude = 40 °.	SOLVED	delta = 48.4392, distance_km = 5386.2

question	outcome	result
Epicentral distance with epicentre latitude = 35.7 °, epicentre longitude = 139.7 ° and station latitude = 51.5 °.	NEEDS_INPUT	epicentral_distance needs lon2 (longitude of the second point (station)). Example: solve('epicentral_distance', lat1=35.
Epicentral distance with epicentre latitude = 35.7 °, epicentre longitude = 139.7 °, station latitude = 51.5 ° and station longitude = -0.1 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Geodesic distance and azimuths on the WGS84 ellipsoid (Vincenty) — `geodesic_distance_azimuth`

Distance along the WGS84 ellipsoid between two points, the azimuth from the first to the second and the back-azimuth, by Vincenty's inverse method.

$$s = bA(\sigma - \Delta\sigma)$$

input	meaning	condition
`lat1`	latitude of the first point (epicentre)	
`lon1`	longitude of the first point (epicentre)	
`lat2`	latitude of the second point (station)	
`lon2`	longitude of the second point (station)	

output	meaning
`distance`	geodesic distance (m)
`azimuth`	azimuth at point 1 (degrees from north)
`back_azimuth`	back-azimuth from point 2 (degrees from north)

- **Method:** Vincenty (1975) inverse iteration on the reduced latitudes
- **Assumes:** WGS84 ellipsoid (a = 6378137 m, f = 1/298.257223563); 0.01 to 120 degrees apart
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('geodesic_distance_azimuth', lat1=35.7, lon1=139.7, lat2=51.5,
lon2=-0.1)
# -> distance = 9.58123e+06 m, azimuth = 336.249, back_azimuth = 31.6632   (SOLVED)
```

```
earth_science_ai999x.solve('geodesic_distance_azimuth', lat1=-33.9, lon1=18.4, lat2=40.7,
lon2=-74.0)
# -> distance = 1.25479e+07 m, azimuth = 304.497, back_azimuth = 115.584   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geodesic distance with epicentre latitude = 35.7 °, epicentre longitude = 139.7 °, station latitude = 51.5 ° and station longitude = -0.1 °.	SOLVED	distance = 9.58123e+06 m, azimuth = 336.249, back_azimuth = 31.6632

question	outcome	result
Geodesic distance with epicentre latitude = -33.9 °, epicentre longitude = 18.4 °, station latitude = 40.7 ° and station longitude = -74 °.	SOLVED	distance = 1.25479e+07 m, azimuth = 304.497, back_azimuth = 115.584
Geodesic distance with epicentre latitude = 35.7 °, epicentre longitude = 139.7 ° and station latitude = 51.5 °.	NEEDS_INPUT	geodesic_distance_azimuth needs lon2 (longitude of the second point (station)). Example: solve('geodesic_distance_azimut
Geodesic distance with epicentre latitude = 35.7 °, epicentre longitude = 139.7 °, station latitude = 51.5 ° and station longitude = -0.1 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Moment magnitude from seismic moment — `moment_magnitude`

Moment magnitude Mw of an earthquake from its scalar seismic moment (Hanks and Kanamori 1979, as ObsPy: $Mw = log10$(M0 in dyne cm)/1.5 - 10.7).

$$M_w = \tfrac{2}{3}\log_{10} M_0[\mathrm{dyn\,cm}] - 10.7$$

input	meaning	condition
`M0`	scalar seismic moment	

output	meaning
`Mw`	moment magnitude
`M0_dyne_cm`	seismic moment (dyne cm)

- **Method:** $Mw = (2/3)$ log10(M0 [dyne cm]) - 10.7
- **Assumes:** $1e6 \le M_0 \le 1e25$ N m
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('moment_magnitude', M0=1.1e+21)
# -> Mw = 7.99426, M0_dyne_cm = 1.1e+28   (SOLVED)
```

```
earth_science_ai999x.solve('moment_magnitude', M0=3.5e+17)
# -> Mw = 5.66271, M0_dyne_cm = 3.5e+24   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moment magnitude with seismic moment = 1.1e+21 N*m.	SOLVED	Mw = 7.99426, M0_dyne_cm = 1.1e+28
Moment magnitude with seismic moment = 3.5e+17 N*m.	SOLVED	Mw = 5.66271, M0_dyne_cm = 3.5e+24
Moment magnitude with seismic moment = 1.1e+21	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

question	outcome	result
N*m and colour = 3.		
Moment magnitude with seismic moment = -5 N*m.	INVALID_INPUT	Moment magnitude from seismic moment: the seismic moment must be between 1e6 and 1e25 N m.

Local (Richter) magnitude from Wood-Anderson amplitude — `local_magnitude`

Local magnitude from the zero-to-peak Wood-Anderson amplitude and the hypocentral distance, with the attenuation of Bakun and Joyner (1984), as ObsPy.

$$M_L = \log_{10} A + \log_{10} \frac{r}{100} + 0.00301\,(r - 100) + 3.0$$

input	meaning	condition
`A`	zero-to-peak Wood-Anderson amplitude	
`r`	hypocentral distance	

output	meaning
`ML`	local magnitude

- **Method:** $ML = log10\ A + log10$(r/100) + 0.00301 $(r - 100) + 3.0$ (A in mm, r in km)
- **Assumes:** 10 $km \leq r \leq 600$ km; calibration of central California (Bakun and Joyner 1984)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('local_magnitude', A=0.01, r=100000.0)
# -> ML = 4   (SOLVED)
```

```
earth_science_ai999x.solve('local_magnitude', A=0.0005, r=50000.0)
# -> ML = 2.24744   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Local magnitude with amplitude = 10 mm and distance = 100 km.	SOLVED	ML = 4
Local magnitude with amplitude = 0.5 mm and distance = 50 km.	SOLVED	ML = 2.24744
Local magnitude with amplitude = 10 mm.	NEEDS_INPUT	local_magnitude needs r (hypocentral distance). Example: solve('local_magnitude', A='0.01 m', r='100000 m').
Local magnitude with amplitude = 10 mm and distance = 100 km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Local magnitude with amplitude = 10 mm and distance = 2 km.	INVALID_INPUT	Local (Richter) magnitude from Wood-Anderson amplitude: the hypocentral distance must be between 10 and 600 km.

Gutenberg-Richter b-value (maximum likelihood) — `gutenberg_richter_b_value`

The b-value of the Gutenberg-Richter law log10 $N = a - bM$ by Aki's maximum-likelihood estimator, with Utsu's correction for magnitudes binned at width dM; also the a-value of the cumulative count at Mc.

$$b = \frac{\log_{10} e}{\overline{M} - (M_c - \Delta M/2)}$$

input	meaning	condition
`mags`	list of event magnitudes	
`Mc`	magnitude of completeness	
`dM`	magnitude bin width (0 for unbinned magnitudes; default 0)	

output	meaning
`b`	b-value
`a`	a-value (log10 of the number of events >= Mc, plus b Mc)
`n`	number of events used
`mean`	mean magnitude of the events used

- **Method:** $b = log10$(e)/(mean(M) - (Mc - dM/2)) (Aki 1965, Utsu 1966)
- **Assumes:** at least 10 events with $M \geq Mc$; magnitudes complete above Mc
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('gutenberg_richter_b_value', mags=[2.1, 2.4, 2.0, 3.1, 2.6, 2.2, 2.9,
2.3, 2.0, 4.0, 2.5, 2.1], Mc=2.0, dM=0.1)
# -> b = 0.766402, a = 2.61199, n = 12, mean = 2.51667   (SOLVED)
```

```
earth_science_ai999x.solve('gutenberg_richter_b_value', mags=[3.0, 3.4, 3.1, 3.9, 3.2, 3.0, 3.6,
3.3, 3.1, 4.4, 3.5], Mc=3.0, dM=0.1)
# -> b = 0.945988, a = 3.87936, n = 11, mean = 3.40909   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
B value with magnitudes = [2.1, 2.4, 2, 3.1, 2.6, 2.2, 2.9, 2.3, 2, 4, 2.5, 2.1], completeness magnitude = 2 and bin width = 0.1.	SOLVED	b = 0.766402, a = 2.61199, n = 12, mean = 2.51667
B value with magnitudes = [3, 3.4, 3.1, 3.9, 3.2, 3, 3.6, 3.3, 3.1, 4.4, 3.5], completeness magnitude = 3 and bin width = 0.1.	SOLVED	b = 0.945988, a = 3.87936, n = 11, mean = 3.40909
B value with magnitudes = [2.1, 2.4, 2, 3.1, 2.6, 2.2, 2.9, 2.3, 2, 4, 2.5, 2.1] and bin width = 0.1.	NEEDS_INPUT	gutenberg_richter_b_value needs Mc (magnitude of completeness). Example: solve('gutenberg_richter_b_value', mags=[2.1, 2
B value with magnitudes = [2.1, 2.4, 2, 3.1, 2.6, 2.2, 2.9, 2.3, 2, 4, 2.5, 2.1],	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
completeness magnitude = 2 and bin width = 0.1 and colour = 3.		earth_science_ai999x.templates();

Normal-moveout velocity from reflection picks — `nmo_velocity`

Stacking (NMO) velocity, zero-offset time and reflector depth from reflection travel times at several offsets, by a least-squares fit of $t^2 = t_0^2 + x^2/V^2$.

$$t^2 = t_0^2 + \frac{x^2}{V_{\mathrm{nmo}}^2}$$

input	meaning	condition
`x`	source-receiver offsets (list, in metres)	
`t`	two-way reflection times (list, in seconds)	

output	meaning
`V_nmo`	NMO velocity (m/s)
`t0`	zero-offset two-way time (s)
`depth`	reflector depth V $t_0/2$ for a constant velocity (m)

- **Method:** least squares of t^2 against x^2: slope $1/V^2$, intercept t_0^2
- **Assumes:** flat reflector, constant velocity above it; offsets in m, times in s
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('nmo_velocity', x=[0, 500, 1000, 1500, 2000], t=[1.0, 1.0198, 1.077,
1.1662, 1.2806])
# -> V_nmo = 2500.07 m/s, t0 = 0.999995 s, depth = 1250.03 m   (SOLVED)
```

```
earth_science_ai999x.solve('nmo_velocity', x=[200, 800, 1600], t=[0.62, 0.71, 0.93])
# -> V_nmo = 2294.14 m/s, t0 = 0.615844 s, depth = 706.418 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nmo velocity with offsets = [0, 500, 1000, 1500, 2000] and times = [1, 1.0198, 1.077, 1.1662, 1.2806].	SOLVED	V_nmo = 2500.07 m/s, t0 = 0.999995 s, depth = 1250.03 m
Nmo velocity with offsets = [200, 800, 1600] and times = [0.62, 0.71, 0.93].	SOLVED	V_nmo = 2294.14 m/s, t0 = 0.615844 s, depth = 706.418 m
Nmo velocity with offsets = [0, 500, 1000, 1500, 2000].	NEEDS_INPUT	nmo_velocity needs t (two-way reflection times (list, in seconds)). Example: solve('nmo_velocity', x=[0, 500, 1000, 1500
Nmo velocity with offsets = [0, 500, 1000, 1500, 2000] and times = [1, 1.0198, 1.077, 1.1662, 1.2806] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Inclination and strength of a geocentric dipole field — `dipole_inclination`

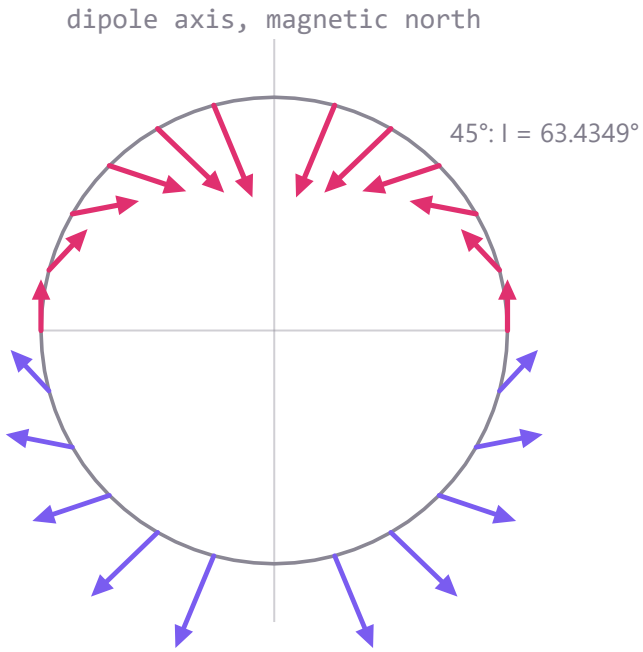
Magnetic inclination (dip) and field strength relative to the equator at a magnetic latitude for a geocentric axial dipole.

$$\tan I = 2\tan\lambda, \quad \frac{F}{F_{eq}} = \sqrt{1+3\sin^2\lambda}$$

input	meaning	condition
`lat`	magnetic latitude (degrees)	

output	meaning
`inclination`	inclination (degrees, positive downward)
`F_ratio`	field strength / equatorial strength

- **Method:** tan $I = 2$ tan(latitude), $F/F_eq = sqrt(1 + 3\ sin^2$ latitude)
- **Assumes:** geocentric axial dipole
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The direction and relative strength of a geocentric dipole field at the surface, drawn on a meridian section of the Earth: at each magnetic latitude from 75° S to 75° N the package returns the inclination (dip below the horizontal) and the field strength relative to the equator, here the arrow's angle and length. Examples: -75°: I = -82.3693°, ×1.94911; -30°: I = -49.1066°, ×1.32288; 15°: I = 28.1868°, ×1.09588; 60°: I = 73.8979°, ×1.80278; at 45° the field dips 63.4349°.

```
earth_science_ai999x.solve('dipole_inclination', lat=45.0)
# -> inclination = 63.4349, F_ratio = 1.58114   (SOLVED)
```

```
earth_science_ai999x.solve('dipole_inclination', lat=-20.0)
# -> inclination = -36.0524, F_ratio = 1.1623   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dipole inclination with magnetic latitude = 45 °.	SOLVED	inclination = 63.4349, F_ratio = 1.58114
Dipole inclination with magnetic latitude = -20 °.	SOLVED	inclination = -36.0524, F_ratio = 1.1623
Dipole inclination with magnetic latitude = 45 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Paleolatitude from magnetic inclination — `paleolatitude`

The magnetic (paleo)latitude at which a rock acquired its magnetisation, from the inclination of its remanence, for a geocentric axial dipole; and the angular distance to the paleopole.

$$\lambda = \arctan\left(\tfrac{1}{2}\tan I\right)$$

input	meaning	condition
`I_deg`	inclination of the remanent magnetisation (degrees)	

output	meaning
`paleolatitude`	paleolatitude (degrees)
`pole_distance`	distance to the pole (degrees)

- **Method:** latitude = atan(tan I/2)
- **Assumes:** geocentric axial dipole; |I| < 90
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('paleolatitude', I_deg=60.0)
# -> paleolatitude = 40.8934, pole_distance = 49.1066   (SOLVED)
```

```
earth_science_ai999x.solve('paleolatitude', I_deg=-30.0)
# -> paleolatitude = -16.1021, pole_distance = 106.102   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Paleolatitude with inclination = 60 °.	SOLVED	paleolatitude = 40.8934, pole_distance = 49.1066
Paleolatitude with inclination = -30 °.	SOLVED	paleolatitude = -16.1021, pole_distance = 106.102
Paleolatitude with inclination = 60 ° and	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

question	outcome	result
colour = 3.		
Paleolatitude with inclination = 95 °.	INVALID_INPUT	Paleolatitude from magnetic inclination: the inclination must be strictly between -90 and 90 degrees.

Plate velocity from an Euler pole — `plate_velocity`

Speed and direction of a point on a rigid plate rotating about an Euler pole (sphere of radius 6371 km).

$$v = \omega R \sin \Delta$$

input	meaning	condition
`pole_lat`	latitude of the Euler pole	
`pole_lon`	longitude of the Euler pole	
`omega`	rotation rate about the pole, counter-clockwise positive (degrees per Myr)	
`lat`	latitude of the site	
`lon`	longitude of the site	

output	meaning
`speed`	plate speed (mm/yr)
`azimuth`	direction of motion (degrees from north)

- **Method:** $v = \omega$ R sin(Delta), direction perpendicular to the great circle to the pole
- **Assumes:** rigid plate on a sphere of radius 6371 km; site 0.5 to 179.5 degrees from the pole
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('plate_velocity', pole_lat=61.5, pole_lon=-73.7, omega=0.75, lat=35.0, lon=-120.0)
# -> speed = 52.794, azimuth = 123.02   (SOLVED)
```

```
earth_science_ai999x.solve('plate_velocity', pole_lat=50.0, pole_lon=-100.0, omega=-0.3, lat=10.0, lon=20.0)
# -> speed = 32.7921, azimuth = 235.508   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Plate velocity with pole latitude = 61.5 °, pole longitude = -73.7 °, rotation rate = 0.75 °/Myr, site latitude = 35 ° and site longitude = -120 °.	SOLVED	speed = 52.794, azimuth = 123.02
Plate velocity with pole latitude = 50 °, pole longitude = -100 °, rotation rate = -0.3 °/Myr, site latitude = 10 ° and site longitude = 20 °.	SOLVED	speed = 32.7921, azimuth = 235.508
Plate velocity with pole latitude = 61.5 °, pole longitude = -73.7 °, rotation rate =	NEEDS_INPUT	plate_velocity needs lon (longitude of the site). Example: solve('plate_velocity', pole_lat=61.5,

question	outcome	result
0.75 °/Myr and site latitude = 35 °.		pole_lon=-73.7, omega=
Plate velocity with pole latitude = 61.5 °, pole longitude = -73.7 °, rotation rate = 0.75 °/Myr, site latitude = 35 ° and site longitude = -120 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Gravity anomaly of an infinite horizontal slab (Bouguer) — `bouguer_slab`

Vertical gravity anomaly of an infinite horizontal slab of thickness h and density contrast drho.

$$\Delta g = 2\pi G\,\Delta\rho\,h$$

input	meaning	condition
`drho`	density contrast of the slab	
`h`	slab thickness	

output	meaning
`dg_mGal`	gravity anomaly (mGal)
`dg`	gravity anomaly (m/s^2)

- **Method:** $dg = 2$ pi G drho h
- **Assumes:** laterally infinite slab; $G = 6.6743e-11$ N m^2/kg^2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('bouguer_slab', drho=2670.0, h=100.0)
# -> dg_mGal = 11.1969, dg = 0.000111969 m/s^2   (SOLVED)
```

```
earth_science_ai999x.solve('bouguer_slab', drho=-400.0, h=2000.0)
# -> dg_mGal = -33.5487, dg = -0.000335487 m/s^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bouguer slab with density contrast = 2670 kg/m^3 and thickness = 100 m.	SOLVED	dg_mGal = 11.1969, dg = 0.000111969 m/s^2
Bouguer slab with density contrast = -400 kg/m^3 and thickness = 2000 m.	SOLVED	dg_mGal = -33.5487, dg = -0.000335487 m/s^2
Bouguer slab with density contrast = 2670 kg/m^3.	NEEDS_INPUT	bouguer_slab needs h (slab thickness). Example: solve('bouguer_slab', drho='2670 kg/m^3', h='100 m').
Bouguer slab with density contrast = 2670 kg/m^3 and thickness = 100 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Gravity anomaly above a buried sphere — `buried_sphere_anomaly`

Peak vertical gravity anomaly directly above a buried sphere of radius R, depth z to its centre and density contrast drho.

$$\Delta g = \frac{4\pi G\,\Delta\rho\,R^3}{3z^2}$$

input	meaning	condition
`drho`	density contrast of the sphere	
`R`	radius of the sphere	
`z`	depth to the centre of the sphere	

output	meaning
`dg_mGal`	peak gravity anomaly (mGal)
`dg`	peak gravity anomaly (m/s^2)

- **Method:** $dg = (4/3)$ pi G drho R^3/z^2
- **Assumes:** $0 < R < z$; observation point directly above the centre
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('buried_sphere_anomaly', drho=500.0, R=100.0, z=200.0)
# -> dg_mGal = 0.349466, dg = 3.49466e-06 m/s^2   (SOLVED)
```

```
earth_science_ai999x.solve('buried_sphere_anomaly', drho=-300.0, R=1000.0, z=3000.0)
# -> dg_mGal = -0.931908, dg = -9.31908e-06 m/s^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Buried sphere with density contrast = 500 kg/m^3, radius = 100 m and depth = 200 m.	SOLVED	dg_mGal = 0.349466, dg = 3.49466e-06 m/s^2
Buried sphere with density contrast = -300 kg/m^3, radius = 1000 m and depth = 3000 m.	SOLVED	dg_mGal = -0.931908, dg = -9.31908e-06 m/s^2
Buried sphere with density contrast = 500 kg/m^3 and radius = 100 m.	NEEDS_INPUT	buried_sphere_anomaly needs z (depth to the centre of the sphere). Example: solve('buried_sphere_anomaly', drho='500 kg/
Buried sphere with density contrast = 500 kg/m^3, radius = 100 m and depth = 200 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Buried sphere with density contrast = 500 kg/m^3, radius = 300 m and depth = 200 m.	INVALID_INPUT	Gravity anomaly above a buried sphere: the sphere must lie wholly below the surface (0 < R < depth of its centre).

Steady geotherm with radiogenic heat production — `geotherm_radiogenic`

Temperature and heat flow at depth in a conducting layer with uniform radiogenic heat production, from the surface temperature and surface heat flow.

$$T(z) = T_0 + \frac{q_0}{k} z - \frac{A}{2k} z^2$$

input	meaning	condition
`T0`	surface temperature	
`q0`	surface heat flow	
`k`	thermal conductivity	
`A`	radiogenic heat production per unit volume	
`z`	depth	

output	meaning
`T`	temperature at depth (K)
`q`	heat flow at depth (W/m^2)

- **Method:** T(z) = T0 + q0 z/k - A $z^2/(2k)$, q(z) = q0 - A z
- **Assumes:** steady one-dimensional conduction; uniform heat production and conductivity
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('geotherm_radiogenic', T0=283.15, q0=0.065, k=2.5, A=1e-06, z=10000.0)
# -> T = 523.15 K, q = 0.055 W/m^2   (SOLVED)
```

```
earth_science_ai999x.solve('geotherm_radiogenic', T0=288.15, q0=0.08, k=3.0, A=2e-06, z=20000.0)
# -> T = 688.15 K, q = 0.04 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geotherm with surface temperature = 10 °C, surface heat flow = 65 mW/m^2, thermal conductivity = 2.5 W/(m*K), heat production = 1 µW/m^3 and depth = 10 km.	SOLVED	T = 523.15 K, q = 0.055 W/m^2
Geotherm with surface temperature = 15 °C, surface heat flow = 80 mW/m^2, thermal conductivity = 3 W/(m*K), heat production = 2 µW/m^3 and depth = 20 km.	SOLVED	T = 688.15 K, q = 0.04 W/m^2
Geotherm with surface temperature = 10 °C, surface heat flow = 65 mW/m^2, thermal conductivity = 2.5 W/(m*K) and heat production = 1 µW/m^3.	NEEDS_INPUT	geotherm_radiogenic needs z (depth). Example: solve('geotherm_radiogenic', T0='283.15 K', q0='0.065 W/m^2', k='2.5 W/(m*
Geotherm with surface temperature = 10 °C, surface heat flow = 65 mW/m^2, thermal conductivity = 2.5 W/(m*K), heat production	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

question	outcome	result
= 1 μW/m^3 and depth = 10 km and colour = 3.		

Depth of the ocean floor with age (half-space cooling) — `seafloor_depth_age`

Seafloor depth as oceanic lithosphere cools and contracts with age (half-space cooling, isostatic compensation under water). Defaults: ridge depth 2500 m, mantle 1300 °C, water 0 °C, alpha $3e-5\ 1/K$, kappa $1e-6\ m^2/s$, rho_m 3300 and rho_w 1000 kg/m^3.

$$d = d_r + \frac{2\rho_m \alpha (T_m - T_w)}{\rho_m - \rho_w}\sqrt{\frac{\kappa t}{\pi}}$$

input	meaning	condition
`t`	age of the seafloor (millions of years)	
`dr`	depth at the ridge (default 2500 m)	
`Tm`	mantle temperature (default 1300 °C)	
`Tw`	seawater temperature (default 0 °C)	
`alpha`	volumetric thermal expansion coefficient (default $3e-5$ 1/K)	
`kappa`	thermal diffusivity (default $1e-6$ m^2/s)	
`rho_m`	mantle density (default 3300 kg/m^3)	
`rho_w`	seawater density (default 1000 kg/m^3)	

output	meaning
`depth`	seafloor depth (m)
`subsidence`	subsidence below the ridge (m)

- **Method:** $d = d_r + 2$ rho_m alpha (Tm - Tw) sqrt(kappa t/pi)/(rho_m - rho_w)
- **Assumes:** 0 < age <= 200 Myr (half-space cooling; old seafloor flattens in reality)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('seafloor_depth_age', t=50.0)
# -> depth = 5008.09 m, subsidence = 2508.09 m   (SOLVED)
```

```
earth_science_ai999x.solve('seafloor_depth_age', t=100.0, dr=2600.0, Tm=1623.15, Tw=275.15,
alpha=3.2e-05, kappa=8e-07, rho_m=3300.0, rho_w=1025.0)
# -> depth = 6147.52 m, subsidence = 3547.52 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Seafloor depth with age = 50 Myr.	SOLVED	depth = 5008.09 m, subsidence = 2508.09 m
Seafloor depth with age = 100 Myr, ridge depth = 2600 m, mantle temperature = 1350 °C, water temperature = 2 °C, thermal expansion = 3.2e-05 1/K, thermal diffusivity = 8e-07 m^2/s,	SOLVED	depth = 6147.52 m, subsidence = 3547.52 m

question	outcome	result
mantle density = 3300 kg/m^3 and water density = 1025 kg/m^3.		
Seafloor depth with age = 50 Myr and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Fraction of a radioactive parent remaining — `radioactive_decay_remaining`

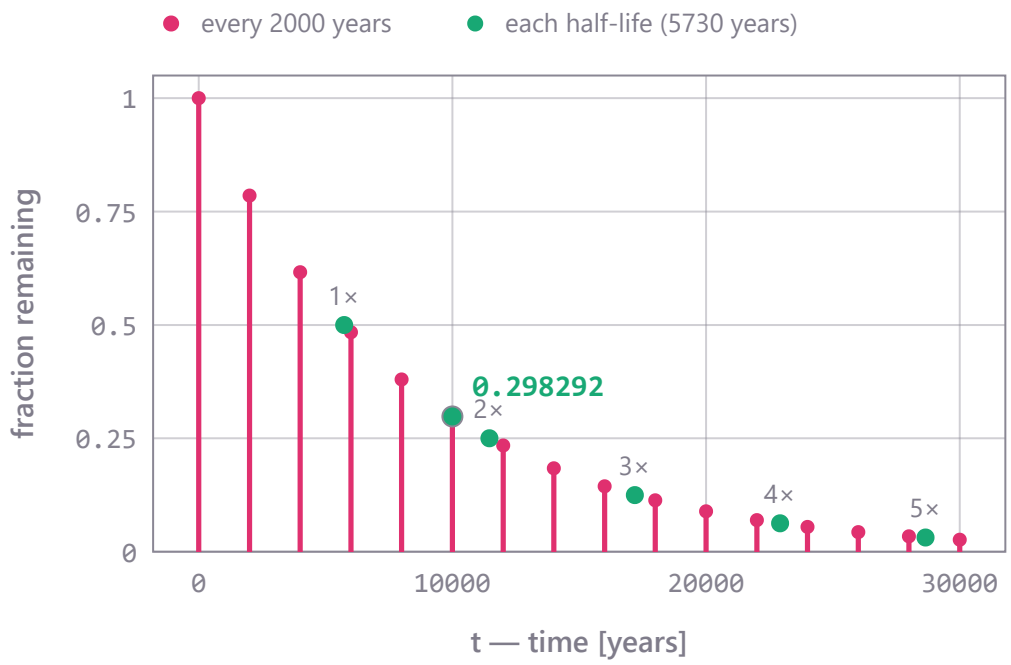
Fraction of the parent nuclide left (and decayed) after a time t, and the decay constant ln 2/half-life.

$$\frac{N}{N_0} = 2^{-t/t_{1/2}} = e^{-\lambda t}, \quad \lambda = \frac{\ln 2}{t_{1/2}}$$

input	meaning	condition
`half_life`	half-life (years)	
`t`	elapsed time (years)	

output	meaning
`fraction`	fraction of the parent remaining
`decayed`	fraction decayed
`lambda`	decay constant $(1/yr)$

- **Method:** N/N0 = 2^(-t/t_half), $\lambda = ln$ 2/t_half
- **Assumes:** half-life > 0; $t \leq 60$ half-lives
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Fraction of carbon-14 remaining (half-life 5730 years) every 2000 years and at each of the first five half-lives, computed by the package (decay constant 0.000120968 per year); a sample with 0.5 of its original carbon-14 is 5568.05 years old. Marked: t = 10000 years, fraction = 0.298292.

```
earth_science_ai999x.solve('radioactive_decay_remaining', half_life=5730.0, t=10000.0)
# -> fraction = 0.298292, decayed = 0.701708, lambda = 0.000120968   (SOLVED)
```

```
earth_science_ai999x.solve('radioactive_decay_remaining', half_life=1250000000.0, t=4500000000.0)
# -> fraction = 0.0824692, decayed = 0.917531, lambda = 5.54518e-10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fraction remaining with half-life = 5730 yr and time = 10000 yr.	SOLVED	fraction = 0.298292, decayed = 0.701708, lambda = 0.000120968
Fraction remaining with half-life = 1.25e+09 yr and time = 4.5e+09 yr.	SOLVED	fraction = 0.0824692, decayed = 0.917531, lambda = 5.54518e-10
Fraction remaining with half-life = 5730 yr.	NEEDS_INPUT	radioactive_decay_remaining needs t (elapsed time (years)). Example: solve('radioactive_decay_remaining', half_life=5730
Fraction remaining with half-life = 5730 yr and time = 10000 yr and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Radiometric age from a daughter/parent ratio — `radiometric_age`

Age of a closed system from the ratio of radiogenic daughter to remaining parent atoms and the half-life (no initial daughter).

$$t = \frac{1}{\lambda}\ln\left(1 + \frac{D^*}{P}\right)$$

input	meaning	condition
`ratio`	radiogenic daughter / parent atom ratio	
`half_life`	half-life of the parent (years)	

output	meaning
`age`	age (years)
`lambda`	decay constant $(1/yr)$

- **Method:** $t = ln$(1 + D/P)/lambda
- **Assumes:** closed system; no daughter at the start
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('radiometric_age', ratio=0.5, half_life=1250000000.0)
# -> age = 7.31203e+08, lambda = 5.54518e-10   (SOLVED)
```

```
earth_science_ai999x.solve('radiometric_age', ratio=3.0, half_life=4468000000.0)
# -> age = 8.936e+09, lambda = 1.55136e-10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radiometric age with daughter to parent ratio = 0.5 and half-life = 1.25e+09 yr.	SOLVED	age = 7.31203e+08, lambda = 5.54518e-10
Radiometric age with daughter to parent ratio = 3 and half-life = 4.468e+09 yr.	SOLVED	age = 8.936e+09, lambda = 1.55136e-10
Radiometric age with daughter to parent ratio = 0.5.	NEEDS_INPUT	radiometric_age needs half_life (half-life of the parent (years)). Example: solve('radiometric_age', ratio=0.5, half_lif
Radiometric age with daughter to parent ratio = 0.5 and half-life = 1.25e+09 yr and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Potassium-argon age (branched decay) — `potassium_argon_age`

K-Ar age from the radiogenic 40Ar*/40K ratio, allowing for the branching of 40K decay to 40Ar (electron capture) and 40Ca. Defaults: $\lambda = 5.543e-10/yr$, $lambda_e = 0.581e-10/yr$ (Steiger and Jäger 1977).

$$t = \frac{1}{\lambda}\ln\left(1 + \frac{\lambda}{\lambda_e}\frac{^{40}\mathrm{Ar}^*}{^{40}\mathrm{K}}\right)$$

input	meaning	condition
`ratio`	radiogenic 40Ar*/40K atom ratio	
`lam`	total decay constant of 40K (default 5.543e-10 /yr)	
`lam_e`	partial decay constant to 40Ar (default 0.581e-10 /yr)	

output	meaning
`age`	K-Ar age (years)

- **Method:** $t = (1/\lambda)$ ln$(1 + (\lambda/lambda_e)$ Ar*/K)
- **Assumes:** closed system, no initial argon
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('potassium_argon_age', ratio=0.0055)
# -> age = 9.22643e+07   (SOLVED)
```

```
earth_science_ai999x.solve('potassium_argon_age', ratio=0.12, lam=5.543e-10, lam_e=5.81e-11)
# -> age = 1.37664e+09   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Potassium argon age with ar/k ratio = 0.0055.	SOLVED	age = 9.22643e+07
Potassium argon age with ar/k ratio = 0.12, decay constant = 5.543e-10 1/yr	SOLVED	age = 1.37664e+09

question	outcome	result
and electron capture constant = 5.81e-11 1/yr.		
Potassium argon age with ar/k ratio = 0.0055 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Isochron age (e.g. Rb-Sr) from sample ratios — `isochron_age`

Age and initial ratio from an isochron: a least-squares line through (parent/stable, daughter/stable) ratios of cogenetic samples (ordinary least squares, not a York fit). Default decay constant $1.42e-11/yr$ (87Rb).

$$t = \frac{\ln(1+m)}{\lambda}, \quad \frac{D}{S} = \left(\frac{D}{S}\right)_0 + m\,\frac{P}{S}$$

input	meaning	condition
`x`	parent/stable isotope ratios (list, e.g. 87Rb/86Sr)	
`y`	daughter/stable isotope ratios (list, e.g. 87Sr/86Sr)	
`lam`	decay constant of the parent (default 1.42e-11 /yr)	

output	meaning
`age`	isochron age (years)
`slope`	isochron slope e^(lambda t) - 1
`initial_ratio`	initial daughter/stable ratio (intercept)

- **Method:** $t = ln$(1 + slope)/lambda
- **Assumes:** cogenetic samples, closed systems, same initial ratio
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('isochron_age', x=[0.5, 1.2, 2.8, 4.1], y=[0.7121, 0.7221, 0.7449,
0.7634], lam=1.42e-11)
# -> age = 9.96372e+08, slope = 0.014249, initial_ratio = 0.70499   (SOLVED)
```

```
earth_science_ai999x.solve('isochron_age', x=[0.2, 0.9, 1.7, 3.3, 5.0], y=[0.70452, 0.71354,
0.72384, 0.74444, 0.76632])
# -> age = 9.00877e+08, slope = 0.0128746, initial_ratio = 0.70195   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Isochron age with parent ratios = [0.5, 1.2, 2.8, 4.1], daughter ratios = [0.7121, 0.7221, 0.7449, 0.7634] and decay constant = 1.42e-11 1/yr.	SOLVED	age = 9.96372e+08, slope = 0.014249, initial_ratio = 0.70499
Isochron age with parent ratios = [0.2, 0.9, 1.7, 3.3, 5] and daughter ratios = [0.70452, 0.71354, 0.72384, 0.74444, 0.76632].	SOLVED	age = 9.00877e+08, slope = 0.0128746, initial_ratio = 0.70195

question	outcome	result
Isochron age with parent ratios = [0.5, 1.2, 2.8, 4.1] and decay constant = 1.42e-11 1/yr.	NEEDS_INPUT	isochron_age needs y (daughter/stable isotope ratios (list, e.g. 87Sr/86Sr)). Example: solve('isochron_age', x=[0.5, 1.2
Isochron age with parent ratios = [0.5, 1.2, 2.8, 4.1], daughter ratios = [0.7121, 0.7221, 0.7449, 0.7634] and decay constant = 1.42e-11 1/yr and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();

Conventional radiocarbon age — `radiocarbon_age`

Conventional radiocarbon age (years BP) from the fraction modern carbon, with the Libby mean life of 8033 years (Stuiver and Polach 1977); not calibrated to calendar years.

$$t = -8033 \ln F$$

input	meaning	condition
F	fraction modern (14C activity relative to the modern standard; or % in a question)	

output	meaning
age	conventional radiocarbon age (years BP)

- **Method:** $t = -8033$ ln F
- **Assumes:** $1e-5 \leq F \leq 1$ (no bomb carbon)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('radiocarbon_age', F=0.5)
# -> age = 5568.05   (SOLVED)
```

```
earth_science_ai999x.solve('radiocarbon_age', F=0.25)
# -> age = 11136.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radiocarbon age with fraction modern = 0.5.	SOLVED	age = 5568.05
Radiocarbon age with fraction modern = 0.25.	SOLVED	age = 11136.1
Radiocarbon age with fraction modern = 0.5 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Radiocarbon age with fraction modern = 1.3.	INVALID_INPUT	Conventional radiocarbon age: the fraction modern must be between 1e-5 and 1.

Settling velocity of a small grain (Stokes) — `stokes_settling_velocity`

Terminal settling velocity of a small sphere in a viscous fluid (Stokes' law), with the grain Reynolds number (refused when it exceeds 1).

$$w = \frac{(\rho_s - \rho_f)\, g\, d^2}{18\, \mu}$$

input	meaning	condition
`d`	grain diameter	
`rho_s`	grain density	
`rho_f`	fluid density	
`mu`	dynamic viscosity of the fluid	

output	meaning
`w`	settling velocity (m/s)
`Re`	grain Reynolds number

- **Method:** $w = (rho_s - rho_f)$ g d^2/(18 mu)
- **Assumes:** grain Reynolds number <= 1; a smooth sphere, no neighbouring grains
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('stokes_settling_velocity', d=5e-05, rho_s=2650.0, rho_f=1000.0,
mu=0.001)
# -> w = 0.00224736 m/s, Re = 0.112368   (SOLVED)
```

```
earth_science_ai999x.solve('stokes_settling_velocity', d=1e-05, rho_s=2650.0, rho_f=1025.0,
mu=0.0012)
# -> w = 7.37769e-05 m/s, Re = 0.000630178   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stokes settling velocity with diameter = 0.05 mm, grain density = 2650 kg/m^3, fluid density = 1000 kg/m^3 and viscosity = 0.001 Pa*s.	SOLVED	w = 0.00224736 m/s, Re = 0.112368
Stokes settling velocity with diameter = 0.01 mm, grain density = 2650 kg/m^3, fluid density = 1025 kg/m^3 and viscosity = 0.0012 Pa*s.	SOLVED	w = 7.37769e-05 m/s, Re = 0.000630178
Stokes settling velocity with diameter = 0.05 mm, grain density = 2650 kg/m^3 and fluid density = 1000 kg/m^3.	NEEDS_INPUT	stokes_settling_velocity needs mu (dynamic viscosity of the fluid). Example: solve('stokes_settling_velocity', d='5e-05
Stokes settling velocity with diameter = 0.05 mm, grain density = 2650 kg/m^3, fluid density = 1000 kg/m^3 and viscosity = 0.001 Pa*s and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Stokes settling velocity with diameter = 0.5 mm, grain density = 2650 kg/m^3, fluid density = 1000 kg/m^3 and viscosity = 0.001 Pa*s.	INVALID_INPUT	Settling velocity of a small grain (Stokes): the grain Reynolds number exceeds 1: Stokes' law does not apply.

Radiative forcing of a change in CO2 (Myhre et al. 1998) — `co2_radiative_forcing`

The radiative forcing $dF = 5.35$ ln(C/C0) W/m^2 of a change in CO2 concentration from C0 to C, and with a climate sensitivity parameter the equilibrium warming lambda dF.

$$\Delta F = 5.35 \ln \frac{C}{C_0} \ \mathrm{W\,m^{-2}}$$

input	meaning	condition
`C`	CO2 concentration (ppm)	
`C0`	reference CO2 concentration (ppm)	
`lam`	climate sensitivity parameter (K per W/m^2, optional)	

output	meaning
`dF`	radiative forcing (W/m^2)
`doublings`	number of CO2 doublings log2(C/C0)
`dT_eq`	equilibrium warming lambda dF (K)

- **Method:** $dF = 5.35$ ln(C/C0) W/m^2; $dT_eq = \lambda$ dF
- **Assumes:** $1 \leq C, C_0 \leq 100$,000 ppm; the simplified logarithmic expression of Myhre et al. (1998)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('co2_radiative_forcing', C=560.0, C0=280.0)
# -> dF = 3.70834 W/m^2, doublings = 1   (SOLVED)
```

```
earth_science_ai999x.solve('co2_radiative_forcing', C=420.0, C0=280.0, lam=0.8)
# -> dF = 2.16924 W/m^2, doublings = 0.584963, dT_eq = 1.73539 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Carbon dioxide radiative forcing with co2 = 560 ppm and reference concentration = 280 ppm.	SOLVED	dF = 3.70834 W/m^2, doublings = 1
Carbon dioxide radiative forcing with co2 = 420 ppm, reference concentration = 280 ppm and sensitivity = 0.8 K/(W/m^2).	SOLVED	dF = 2.16924 W/m^2, doublings = 0.584963, dT_eq = 1.73539 K
Carbon dioxide radiative forcing with co2 = 560 ppm.	NEEDS_INPUT	co2_radiative_forcing needs C0 (reference CO2 concentration (ppm)). Example: solve('co2_radiative_forcing', C=560, C0=28
Carbon dioxide radiative forcing with co2 = 560 ppm and reference concentration = 280 ppm and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Carbon dioxide radiative forcing with co2 = -5 ppm and reference	INVALID_INPUT	Radiative forcing of a change in CO2 (Myhre et al. 1998): CO2 concentrations must be between 1 and

question	outcome	result
concentration = 280 ppm.		100,000 ppm.

Distance to an earthquake from the S-P interval — `sp_epicentral_distance`

The distance from a station to the hypocentre from the delay between the S and P arrivals, for straight rays at constant P and S speeds.

$$d = \frac{\Delta t \, v_p v_s}{v_p - v_s}$$

input	meaning	condition
`dt`	S minus P arrival time	
`vp`	P-wave speed	
`vs`	S-wave speed	

output	meaning
`distance`	distance to the source (m)
`distance_km`	distance (km)
`tp`	P travel time (s)
`ts`	S travel time (s)

- **Method:** $d = dt$ vp vs/(vp - vs)
- **Assumes:** $vp > vs > 0$; straight rays at constant speeds (a local, crustal distance)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('sp_epicentral_distance', dt=10.0, vp=6000.0, vs=3500.0)
# -> distance = 84000 m, distance_km = 84, tp = 14 s, ts = 24 s   (SOLVED)
```

```
earth_science_ai999x.solve('sp_epicentral_distance', dt=45.0, vp=8000.0, vs=4500.0)
# -> distance = 462857 m, distance_km = 462.857, tp = 57.8571 s, ts = 102.857 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
S p epicentral distance with s-p interval = 10 s, p wave speed = 6 km/s and s wave speed = 3.5 km/s.	SOLVED	distance = 84000 m, distance_km = 84, tp = 14 s, ts = 24 s
S p epicentral distance with s-p interval = 45 s, p wave speed = 8 km/s and s wave speed = 4.5 km/s.	SOLVED	distance = 462857 m, distance_km = 462.857, tp = 57.8571 s, ts = 102.857 s
S p epicentral distance with s-p interval = 10 s and p wave speed = 6 km/s.	NEEDS_INPUT	sp_epicentral_distance needs vs (S-wave speed). Example: solve('sp_epicentral_distance', dt='10 s', vp='6000 m/s', vs='3

question	outcome	result
S p epicentral distance with s-p interval = 10 s, p wave speed = 6 km/s and s wave speed = 3.5 km/s and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
S p epicentral distance with s-p interval = 10 s, p wave speed = 3 km/s and s wave speed = 3.5 km/s.	INVALID_INPUT	Distance to an earthquake from the S-P interval: need vp > vs > 0.

Mountain root by Airy isostasy — `airy_isostasy`

The crustal root below topography of height h compensated by Airy isostasy (crust of uniform density floating on denser mantle), and with a normal crustal thickness the total thickness under the mountain.

$$r = \frac{\rho_c}{\rho_m - \rho_c} h$$

input	meaning	condition
`h`	elevation of the topography above sea level	
`rho_c`	crustal density	
`rho_m`	mantle density	
`tc`	normal crustal thickness (optional)	

output	meaning
`root`	root depth below the normal crust (m)
`root_km`	root depth (km)
`crust_thickness`	total crustal thickness under the topography (m)

- **Method:** $r = h$ rho_c/(rho_m - rho_c)
- **Assumes:** $h > 0$; $rho_m > rho_c$; local (Airy) compensation, uniform crustal density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('airy_isostasy', h=3000.0, rho_c=2800.0, rho_m=3300.0)
# -> root = 16800 m, root_km = 16.8   (SOLVED)
```

```
earth_science_ai999x.solve('airy_isostasy', h=5000.0, rho_c=2700.0, rho_m=3300.0, tc=35000.0)
# -> root = 22500 m, root_km = 22.5, crust_thickness = 62500 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Airy isostasy with elevation = 3 km, crustal density = 2800 kg/m^3 and mantle density = 3300 kg/m^3.	SOLVED	root = 16800 m, root_km = 16.8
Airy isostasy with elevation = 5 km, crustal density = 2700 kg/m^3, mantle	SOLVED	root = 22500 m, root_km = 22.5, crust_thickness = 62500 m

question	outcome	result
density = 3300 kg/m^3 and crustal thickness = 35 km.		
Airy isostasy with elevation = 3 km and crustal density = 2800 kg/m^3.	NEEDS_INPUT	airy_isostasy needs rho_m (mantle density). Example: solve('airy_isostasy', h='3000 m', rho_c='2800 kg/m^3', rho_m='3300
Airy isostasy with elevation = 3 km, crustal density = 2800 kg/m^3 and mantle density = 3300 kg/m^3 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Airy isostasy with elevation = 3 km, crustal density = 3400 kg/m^3 and mantle density = 3300 kg/m^3.	INVALID_INPUT	Mountain root by Airy isostasy: need rho_m > rho_c > 0.

Column density by Pratt isostasy — `pratt_isostasy`

The density of a crustal column standing h above sea level under Pratt isostasy (columns of different density down to a common compensation depth D), given the density of a column whose top is at sea level.

$$\rho = \rho_0 \frac{D}{D+h}$$

input	meaning	condition
`h`	elevation of the column above sea level	
`rho0`	density of a column whose top is at sea level	
`D`	depth of compensation below sea level	

output	meaning
`rho`	density of the elevated column (kg/m^3)
`drho`	density deficit $rho0 - \rho\ (kg/m^3)$

- **Method:** $\rho = rho0$ D/(D + h)
- **Assumes:** $h > 0$; rho0, $D > 0$; a column on land (no water load)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('pratt_isostasy', h=3000.0, rho0=2800.0, D=100000.0)
# -> rho = 2718.45 kg/m^3, drho = 81.5534 kg/m^3   (SOLVED)
```

```
earth_science_ai999x.solve('pratt_isostasy', h=1000.0, rho0=2900.0, D=80000.0)
# -> rho = 2864.2 kg/m^3, drho = 35.8025 kg/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pratt isostasy with elevation = 3 km, reference density = 2800 kg/m^3 and compensation depth = 100 km.	SOLVED	rho = 2718.45 kg/m^3, drho = 81.5534 kg/m^3

question	outcome	result
Pratt isostasy with elevation = 1 km, reference density = 2900 kg/m^3 and compensation depth = 80 km.	SOLVED	rho = 2864.2 kg/m^3, drho = 35.8025 kg/m^3
Pratt isostasy with elevation = 3 km and reference density = 2800 kg/m^3.	NEEDS_INPUT	pratt_isostasy needs D (depth of compensation below sea level). Example: solve('pratt_isostasy', h='3000 m', rho0='2800
Pratt isostasy with elevation = 3 km, reference density = 2800 kg/m^3 and compensation depth = 100 km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Pratt isostasy with elevation = -3 km, reference density = 2800 kg/m^3 and compensation depth = 100 km.	INVALID_INPUT	Column density by Pratt isostasy: the elevation must be positive.

Gradient wind in curved flow — `gradient_wind`

The gradient wind speed of curved, frictionless flow from the geostrophic wind, the radius of curvature of the trajectory and the latitude, for cyclonic or anticyclonic flow (the regular solution).

$$\frac{V^2}{R} \pm fV = fV_g$$

input	meaning	condition
`lat`	latitude (degrees, north positive)	
`R`	radius of curvature of the flow	
`Vg`	geostrophic wind speed	
`flow`	cyclonic or anticyclonic	

output	meaning
`V`	gradient wind speed (m/s)
`ratio`	V/Vg
`f`	|f| $(1/s)$

- **Method:** cyclonic $V = -fR/2 + sqrt$(Double exponent: use braces to clarify R Vg); anticyclonic $V = fR/2 - sqrt$(Double exponent: use braces to clarify R Vg)
- **Assumes:** 5 <= |latitude| <= 85 degrees; R, $Vg > 0$; anticyclonic: Vg <= |f| $R/4$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('gradient_wind', lat=45.0, R=500000.0, Vg=20.0, flow='cyclonic')
# -> V = 15.4004 m/s, ratio = 0.770018, f = 0.000103126 1/s   (SOLVED)
```

```
earth_science_ai999x.solve('gradient_wind', lat=45.0, R=1000000.0, Vg=10.0, flow='anticyclonic')
# -> V = 11.2209 m/s, ratio = 1.12209, f = 0.000103126 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gradient wind with latitude = 45 °, radius = 500 km, geostrophic wind = 20 m/s and flow type = cyclonic.	SOLVED	V = 15.4004 m/s, ratio = 0.770018, f = 0.000103126 1/s
Gradient wind with latitude = 45 °, radius = 1000 km, geostrophic wind = 10 m/s and flow type = anticyclonic.	SOLVED	V = 11.2209 m/s, ratio = 1.12209, f = 0.000103126 1/s
Gradient wind with latitude = 45 °, radius = 500 km and geostrophic wind = 20 m/s.	NEEDS_INPUT	gradient_wind needs flow (cyclonic or anticyclonic). Example: solve('gradient_wind', lat=45, R='500000 m', Vg='20 m/s',
Gradient wind with latitude = 45 °, radius = 500 km, geostrophic wind = 20 m/s and flow type = cyclonic and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Gradient wind with latitude = 45 °, radius = 100 km, geostrophic wind = 30 m/s and flow type = anticyclonic.	INVALID_INPUT	Gradient wind in curved flow: no regular anticyclonic gradient wind: the geostrophic wind exceeds f R/4.

Water-wave dispersion at finite depth — `wave_dispersion`

Wavelength, phase speed and group speed of a linear surface gravity wave of period T in water of depth h, from the full dispersion relation $\omega^2 = g$ k tanh(k h) (deep and shallow limits included).

$$\omega^2 = gk\tanh(kh), \quad c_g = \frac{c}{2}\left(1 + \frac{2kh}{\sinh 2kh}\right)$$

input	meaning	condition
`T`	wave period	
`h`	water depth	

output	meaning
`L`	wavelength (m)
`c`	phase speed (m/s)
`cg`	group speed (m/s)
`kh`	relative depth k h
`L0`	deep-water wavelength g T^2/(2 pi) (m)

- **Method:** $\omega^2 = g$ k tanh(k h) solved by Newton's method; $c = \omega/k$; $c_g = c$ (1 + 2kh/sinh 2kh)/2
- **Assumes:** $0.5 \le T \le 60$ s; $0 < h \le 11$,000 m; linear (small-amplitude) waves
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('wave_dispersion', T=10.0, h=20.0)
# -> L = 121.21 m, c = 12.121 m/s, cg = 9.27161 m/s, kh = 1.03675, L0 = 156.078 m   (SOLVED)
```

```
earth_science_ai999x.solve('wave_dispersion', T=6.0, h=100.0)
# -> L = 56.188 m, c = 9.36466 m/s, cg = 4.68233 m/s, kh = 11.1824, L0 = 56.188 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Finite depth wave with period = 10 s and depth = 20 m.	SOLVED	L = 121.21 m, c = 12.121 m/s, cg = 9.27161 m/s, kh = 1.03675, L0 = 156.078 m
Finite depth wave with period = 6 s and depth = 100 m.	SOLVED	L = 56.188 m, c = 9.36466 m/s, cg = 4.68233 m/s, kh = 11.1824, L0 = 56.188 m
Finite depth wave with period = 10 s.	NEEDS_INPUT	wave_dispersion needs h (water depth). Example: solve('wave_dispersion', T='10 s', h='20 m').
Finite depth wave with period = 10 s and depth = 20 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Finite depth wave with period = 10 s and depth = -5 m.	INVALID_INPUT	Water-wave dispersion at finite depth: the depth must be between 0 and 11,000 m.

Groundwater discharge by Darcy's law — `darcy_flow`

The volumetric discharge $Q = K$ A dh/L through a porous medium, the specific discharge (Darcy flux) and, with a porosity, the average linear (seepage) velocity.

$$Q = KA\,\frac{\Delta h}{L}$$

input	meaning	condition
`K`	hydraulic conductivity	
`A`	cross-sectional area of flow	
`dh`	head difference	
`L`	flow length	
`n`	effective porosity (optional)	

output	meaning
`Q`	discharge (m^3/s)
`q`	specific discharge (Darcy flux, m/s)
`gradient`	hydraulic gradient dh/L
`v`	average linear velocity q/n (m/s)

- **Method:** $Q = K$ A dh/L, $q = Q/A$, $v = q/n$
- **Assumes:** K, A, $L > 0$; $dh > 0$; laminar flow in a saturated, homogeneous medium
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('darcy_flow', K=0.0001, A=50.0, dh=2.0, L=100.0)
# -> Q = 0.0001 m^3/s, q = 2e-06 m/s, gradient = 0.02   (SOLVED)
```

```
earth_science_ai999x.solve('darcy_flow', K=5e-05, A=20.0, dh=5.0, L=250.0, n=0.3)
# -> Q = 2e-05 m^3/s, q = 1e-06 m/s, gradient = 0.02, v = 3.33333e-06 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Darcy flow with hydraulic conductivity = 0.0001 m/s, area = 50 m^2, head difference = 2 m and length = 100 m.	SOLVED	Q = 0.0001 m^3/s, q = 2e-06 m/s, gradient = 0.02
Darcy flow with hydraulic conductivity = 5e-05 m/s, area = 20 m^2, head difference = 5 m, length = 250 m and porosity = 0.3.	SOLVED	Q = 2e-05 m^3/s, q = 1e-06 m/s, gradient = 0.02, v = 3.33333e-06 m/s
Darcy flow with hydraulic conductivity = 0.0001 m/s, area = 50 m^2 and head difference = 2 m.	NEEDS_INPUT	darcy_flow needs L (flow length). Example: solve('darcy_flow', K='0.0001 m/s', A='50 m^2', dh='2 m', L='100 m').
Darcy flow with hydraulic conductivity = 0.0001 m/s, area = 50 m^2, head difference = 2 m and length = 100 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Darcy flow with hydraulic conductivity = 0.0001 m/s, area = -50 m^2, head difference = 2 m and length = 100 m.	INVALID_INPUT	Groundwater discharge by Darcy's law: K, A and L must be positive.

Stream power of a river — `stream_power`

The total stream power per unit channel length $Omega = \rho$ g Q S and, with the channel width, the unit stream power $\omega = Omega/w$.

$$\Omega = \rho g Q S, \quad \omega = \frac{\Omega}{w}$$

input	meaning	condition
`Q`	discharge	
`S`	channel slope (m/m)	
`w`	channel width (optional)	
`rho`	water density (default 1000 kg/m^3)	

output	meaning
`Omega`	total stream power per unit length (W/m)
`omega`	unit stream power (W/m^2)

- **Method:** $Omega = \rho$ g Q S, $\omega = Omega/w$
- **Assumes:** $Q > 0$; $0 < S \leq 1$; steady uniform flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('stream_power', Q=100.0, S=0.002)
# -> Omega = 1961.33 W/m   (SOLVED)
```

```
earth_science_ai999x.solve('stream_power', Q=50.0, S=0.005, w=20.0)
# -> Omega = 2451.66 W/m, omega = 122.583 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stream power with discharge = 100 m^3/s and slope = 0.002.	SOLVED	Omega = 1961.33 W/m
Stream power with discharge = 50 m^3/s, slope = 0.005 and width = 20 m.	SOLVED	Omega = 2451.66 W/m, omega = 122.583 W/m^2
Stream power with discharge = 100 m^3/s.	NEEDS_INPUT	stream_power needs S (channel slope (m/m)). Example: solve('stream_power', Q='100 m^3/s', S=0.002).
Stream power with discharge = 100 m^3/s and slope = 0.002 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Stream power with discharge = 100 m^3/s and slope = 1.5.	INVALID_INPUT	Stream power of a river: the slope must be between 0 and 1 (m/m).

Rossby number — `rossby_number`

The Rossby number $Ro = U/(|f|\ L)$ of a flow with speed scale U and length scale L at a latitude: the ratio of inertial to Coriolis accelerations.

$$Ro = \frac{U}{|f|\,L}$$

input	meaning	condition
`U`	velocity scale	
`L`	horizontal length scale	
`lat`	latitude (degrees, north positive)	

output	meaning
`Ro`	Rossby number
`f`	|f| $(1/s)$

- **Method:** $Ro = U/(|f|\ L)$, $f = 2$ Omega sin(latitude)
- **Assumes:** |latitude| >= 1 degree; U, $L > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('rossby_number', U=10.0, L=1000000.0, lat=45.0)
# -> Ro = 0.0969687, f = 0.000103126 1/s   (SOLVED)
```

```
earth_science_ai999x.solve('rossby_number', U=0.1, L=100000.0, lat=30.0)
# -> Ro = 0.0137134, f = 7.29211e-05 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rossby number with speed = 10 m/s, length scale = 1000 km and latitude = 45 °.	SOLVED	Ro = 0.0969687, f = 0.000103126 1/s
Rossby number with speed = 0.1 m/s, length scale = 100 km and latitude = 30 °.	SOLVED	Ro = 0.0137134, f = 7.29211e-05 1/s
Rossby number with speed = 10 m/s and length scale = 1000 km.	NEEDS_INPUT	rossby_number needs lat (latitude (degrees, north positive)). Example: solve('rossby_number', U='10 m/s', L='1e+06 m', l
Rossby number with speed = 10 m/s, length scale = 1000 km and latitude = 45 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Rossby number with speed = 10 m/s, length scale = 1000 km and latitude = 0 °.	INVALID_INPUT	Rossby number: the Rossby number is not defined at the equator (|latitude| >= 1 degree).

Solar elevation and azimuth — `solar_elevation_angle`

The sun's elevation above the horizon, its zenith angle and azimuth (clockwise from north) for a latitude, a solar declination and an hour angle (15 degrees per hour from solar noon, negative in the morning).

$$\sin\alpha = \sin\varphi\sin\delta + \cos\varphi\cos\delta\cos H$$

input	meaning	condition
`lat`	latitude (degrees, north positive)	
`decl`	solar declination (degrees)	
`H`	hour angle (degrees, negative before solar noon)	

output	meaning
`elevation`	solar elevation angle (degrees)
`zenith`	solar zenith angle (degrees)
`azimuth`	solar azimuth, clockwise from north (degrees)

- **Method:** sin(elevation) = sin(phi) sin(delta) + cos(phi) cos(delta) cos(H); azimuth from the east and north components
- **Assumes:** |declination| <= 23.5 degrees; -180 <= hour angle <= 180 degrees; geometric (no refraction)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('solar_elevation_angle', lat=40.0, decl=23.44, H=0.0)
# -> elevation = 73.44, zenith = 16.56, azimuth = 180   (SOLVED)
```

```
earth_science_ai999x.solve('solar_elevation_angle', lat=40.0, decl=0.0, H=45.0)
# -> elevation = 32.7978, zenith = 57.2022, azimuth = 237.268   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Solar elevation angle with latitude = 40 °, declination = 23.44 ° and hour angle = 0 °.	SOLVED	elevation = 73.44, zenith = 16.56, azimuth = 180
Solar elevation angle with latitude = 40 °, declination = 0 ° and hour angle = 45 °.	SOLVED	elevation = 32.7978, zenith = 57.2022, azimuth = 237.268
Solar elevation angle with latitude = 40 ° and declination = 23.44 °.	NEEDS_INPUT	solar_elevation_angle needs H (hour angle (degrees, negative before solar noon)). Example: solve('solar_elevation_angle'
Solar elevation angle with latitude = 40 °, declination = 23.44 ° and hour angle = 0 ° and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Solar elevation angle with latitude = 40 °, declination = 30 ° and hour angle = 0 °.	INVALID_INPUT	Solar elevation and azimuth: the solar declination must be between -23.5 and 23.5 degrees.

Surface heat flow of cooling oceanic lithosphere (half-space model) — `half_space_heat_flow`

The conductive surface heat flow $q = k$ dT/sqrt(pi kappa t) of oceanic lithosphere of age t (half-space cooling) and its thermal thickness. Defaults: dT 1300 K, k 3.3 W/(m K), kappa $1e-6\ m^2/s$.

$$q = \frac{k\,\Delta T}{\sqrt{\pi \kappa t}}$$

input	meaning	condition
`t`	age of the seafloor (millions of years)	
`dT`	mantle minus surface temperature (default 1300 K)	
`k`	thermal conductivity (default 3.3 W/(m K))	
`kappa`	thermal diffusivity (default $1e-6$ m^2/s)	

output	meaning
`q`	surface heat flow (W/m^2)
`q_mW`	surface heat flow (mW/m^2)
`thickness`	thermal thickness, where $(T - Ts)/dT = 0.9$ (m)

- **Method:** $q = k$ dT/sqrt(pi kappa t); thermal thickness 2 erfinv(0.9) sqrt(kappa t) = 2.326 sqrt(kappa t)
- **Assumes:** $0 < t \leq 200$ Myr; conductive half-space cooling
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
earth_science_ai999x.solve('half_space_heat_flow', t=50.0)
# -> q = 0.060932 W/m^2, q_mW = 60.932, thickness = 92401.5 m   (SOLVED)
```

```
earth_science_ai999x.solve('half_space_heat_flow', t=10.0, dT=1350.0, k=3.1, kappa=8e-07)
# -> q = 0.148602 W/m^2, q_mW = 148.602, thickness = 36960.6 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Half space heat flow with age = 50 Myr.	SOLVED	q = 0.060932 W/m^2, q_mW = 60.932, thickness = 92401.5 m
Half space heat flow with age = 10 Myr, temperature difference = 1350 K, thermal conductivity = 3.1 W/(m*K) and thermal diffusivity = 8e-07 m^2/s.	SOLVED	q = 0.148602 W/m^2, q_mW = 148.602, thickness = 36960.6 m
Half space heat flow with age = 50 Myr and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: earth_science_ai999x.templates();
Half space heat flow with age = 500 Myr.	INVALID_INPUT	Surface heat flow of cooling oceanic lithosphere (half-space model): the seafloor age must be between 0 and 200 Myr.